



Lehrstuhl für Data Science

Scalable Subgraph-Based Retrieval for Graph-Enhanced RAG Systems

Masterarbeit von

Timur Garipov

1. PRÜFER

2. PRÜFER

Prof. Dr. Michael Granitzer Prof. Dr. Harald Kosch

August 18, 2025

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Research Questions	3
1.3	Contribution of the Thesis	4
1.4	Structure of the Thesis	5
2	Background	9
2.1	Retrieval-Augmented Generation: Concept and Architecture	9
2.1.1	Naive RAG Pipeline	11
2.1.2	Advanced RAG Approaches	12
2.2	Graph-Based Retrieval-Augmented Generation	13
2.2.1	GraphRAG Pipeline	13
2.2.2	Subgraph Retrieval in Knowledge Graphs	14
2.3	Evaluation in Retrieval-Augmented Generation	15
2.4	Optimization in Retrieval-Augmented Generation	16
2.5	Related Work	17
3	Methods	21
3.1	Unified Dataset for Comparative Evaluation	21
3.1.1	Train Set Preparation	22
3.1.2	Test Set Formatting	24
3.1.3	Wikipedia Article Extraction for Baselines	25
3.2	Subgraph-Based RAG Architecture	26
3.2.1	Subgraph Retriever Training	26
3.2.2	Triplet-Based Retrieval	30
3.2.3	Subgraph Construction	31
3.2.4	Pre-Reranking and Pruning	32
3.2.5	Query-Focused Summarization and Answer Prediction	33

4	Results	35
4.1	Experimental setup	35
4.1.1	Development Setup	35
4.1.2	System Setup	36
4.1.3	Dataset	38
4.2	Document-Based Baselines: Naive and Advanced RAG Pipelines	40
4.2.1	Naive RAG Implementation	40
4.2.2	Advanced GraphRAG Baseline: LightRAG Index Construction	41
4.2.3	Answers Generation with LightRAG	42
4.3	Retriever Modeling and Evaluation	43
4.3.1	Preprocessing: Weak Supervision for Relation Selection	43
4.3.2	Training Setup and Hyperparameter Study	45
4.3.3	Retriever Comparison and Selection	48
4.3.4	Beam Width and Depth Sensitivity Analysis	50
4.4	End-to-End Evaluation with RAG	51
4.5	Comparative Evaluation with Baselines	52
5	Discussion	55
5.1	Limitations And Future Work	56
6	Conclusion	59
	Appendix A Code, Artifacts and Dataset	60
	Appendix B Generation Prompt	61
	Appendix C Answer Prompt	62
	Bibliography	63
	Eidesstattliche Erklärung	69

Abstract

Large Language Models (LLMs) have achieved remarkable results in various natural language processing tasks. However, their effectiveness remains limited by the scope of their training data and the constraints of the context window. Retrieval-Augmented Generation (RAG) addresses these limitations by incorporating external knowledge into the generation process. Although most RAG systems rely on unstructured text as their knowledge source, recent research has explored the use of structured data, particularly knowledge graphs (KGs), as a richer and more semantically precise source of context.

This thesis proposes a novel graph-based RAG pipeline for multi-hop open-domain question answering. The approach integrates entity linking with a trained retriever to extract candidate triplets from a knowledge graph, constructs head-centric subgraphs through reranking and compaction, and transforms them into concise textual query-focused summaries. These summaries, grounded in the knowledge graph, are then combined with the question and passed to a language model to generate faithful, evidence-based answers.

To evaluate the effectiveness of this approach, we compare it with multiple baselines, including traditional naive RAG using unstructured documents aligned with the question entities, as well as advanced graph-based retrieval architectures such as LightRAG.

We evaluate performance using answer correctness and complement this with qualitative insights into retrieval relevance and computational efficiency. These findings offer practical guidance for the design of knowledge-augmented generation systems and demonstrate the potential of knowledge graphs for grounded and efficient context in RAG.

Acknowledgments

I would like to thank my thesis advisor, Mehdi Ben Amor, at the Chair of Data Science, University of Passau, for his expert guidance throughout this research. He helped me define and shape the direction of the thesis, provided valuable feedback and suggestions, and supported me with weekly meetings that greatly enhanced the quality and focus of my work. His insights and constructive criticism played a crucial role in the development and refinement of this thesis. I am deeply grateful for his commitment and encouragement.

List of Figures

2.1	A typical RAG pipeline for question answering with three stages: (1) Indexing — segmenting and embedding documents into a vector store; (2) Retrieval — selecting top- k relevant chunks by similarity; (3) Generation — producing an answer using the query and retrieved context.	11
3.1	Workflow for preparing grounded training set for the subgraph retriever .	24
3.2	Illustration of the subgraph retrieving process. Taken from Zhang et al. (2022) [Zha+22].	26
3.3	Overview of the full SubgraphRAG pipeline, including retriever training and the main pipeline stages.	27
4.1	Loss behavior for Jaccard ($n_{\text{neg}}=15, \tau=0.5$)	46
4.2	Loss behavior for Recall ($n_{\text{neg}}=15, \tau=0.5$)	47
4.3	Loss behavior for Jaccard, 1 epoch, $\text{LR} = 10^{-5}$	48
4.4	Loss behavior for Recall, 3 epoch, $\text{LR} = 2 \times 10^{-5}$	48
4.5	Loss behavior for Jaccard 30 negatives 0.4 threshold, 3 epoch, $\text{LR} = 2 \times 10^{-5}$	49

List of Tables

2.1	Summary of evaluation dimensions in RAG	16
2.2	Summary of related retrieval-augmented generation methods	18
3.1	Field mapping from original train dataset partition to cleaned format. . .	23
3.2	Field mapping from original test dataset partition to evaluation-ready format.	25
4.1	Number of valid examples before and after preparing	39
4.2	Wikipedia article retrieval statistics	39
4.3	Preprocessing outcomes.	44
4.4	Training profiles and retrieval performance on the test split (evaluated with beam=6, depth=2).	49
4.5	Beam/depth ablation for two retrievers. M1: Jaccard (15, 0.5), LR= 2×10^{-5} , 9 epochs; M2: Jaccard (30, 0.4), LR= 2×10^{-5} , 3 epochs. Retrieval on the same split.	50
4.6	Configuration of subgraph pruning and query-focused summarization for R-balanced and R-high. Shared parameters are listed jointly; summariza- tion budgets differ across variants.	52
4.7	Final evaluation of our pipeline compared to baselines. We report both the RAGAS-based answer correctness and the proportion of abstentions labeled as NO_ANSWER	53

1 Introduction

In recent years, Large Language Models (LLMs) have become foundational components in natural language processing systems, demonstrating remarkable capabilities across a range of generative and knowledge-intensive tasks [Min+25; Bro+20; Tou+23; Cho+22]. These models, trained on vast corpora of text, have shown proficiency in producing fluent, contextually relevant, and often informative responses. However, reliance on fixed training data introduces inherent limitations - most notably, a lack of access to up-to-date or domain-specific knowledge and a tendency to generate plausible but factually incorrect statements, commonly referred to as hallucinations [Ji+23; Hua+25].

To address these challenges, the *Retrieval-Augmented Generation* (RAG) framework has emerged as a hybrid solution, integrating external knowledge retrieval into the generation process [Lew+21; Guu+20; LCT19]. By allowing LLMs to access relevant documents or data snippets at inference time, RAG enables more grounded output without requiring retraining. While early RAG systems primarily relied on unstructured document collections, more recent approaches have turned to knowledge graphs (KGs) - structured, semantically rich representations of entities and their relationships. This shift has led to the development of GraphRAG - Retrieval Augmented Generation (RAG) using a Knowledge Graph, a promising paradigm that combines the strengths of LLMs, retrieval and structured reasoning [Pen+24a; Hu+24]

1.1 Motivation

While Retrieval-Augmented Generation (RAG) has significantly advanced the ability of Large Language Models (LLMs) to integrate external knowledge, effectively leveraging *structured* sources such as knowledge graphs remains challenging. Knowledge graphs capture entities and explicit semantic relations between them, enabling more precise

reasoning and potentially improving factual accuracy in multi-hop scenarios. However, many existing RAG pipelines struggle to fully exploit these benefits. Key difficulties include retrieving a context that is both relevant and concise, preserving the semantic structure of the graph during generation, and balancing retrieval precision with sufficient coverage.

A common approach to incorporating knowledge graphs into RAG involves an intermediate indexing step, where a searchable representation of the data is constructed in advance [Gao+24]. While this step can enable efficient retrieval, it also introduces design and maintenance considerations — such as the choice of representation (e.g., text chunks, vector embeddings, graph encodings) and the handling of updates over time. In this work, we focus on an alternative setup: starting from an already established large-scale knowledge graph, Wikidata [VK14], and retrieving relevant information directly without building a dedicated index from scratch.

The central retrieval mechanism in this work is *subgraph retrieval*: given a user query, the system dynamically identifies a set of semantically relevant triples (subject – predicate – object facts) from the knowledge graph and returns them as a compact, connected subgraph. We employ a pretrained subgraph retriever originally developed for multi-hop Knowledge Base Question Answering (KBQA) [Zha+22]. This model selects facts that are most likely to contribute to answering the question, effectively narrowing the search space while preserving structural relationships between entities.

A practical challenge in training subgraph retrievers is the scarcity of step-level supervision. In this work, we therefore rely on weak supervision: from question–answer pairs and the underlying knowledge graph, we derive pseudo-labeled multi-hop paths and form training signals as “follow this relation” vs “distractors” at each step. This avoids costly manual annotation but introduces design choices (e.g., how to score and threshold candidate paths, how many negatives to sample) that directly trade off dataset size and noise. As these choices can significantly affect the learned retriever and downstream RAG performance, we explicitly analyze their impact, with the underlying algorithmic procedures detailed in Sec. 3.2.1, and the concrete experimental setups, trained models, and evaluation outcomes reported in Sec. 4.3.1.

Beyond the retrieval step itself, an important but often overlooked dimension in RAG optimisation is the ranking and structuring of the retrieved context. Prior work has shown that the ordering and coherence of retrieved information can significantly affect

the generator’s ability to integrate it into accurate answers [Lew+21]. Moreover, practical deployments must account for the limited context windows of LLMs — often only a few thousand tokens for smaller or more efficient models — which makes it infeasible to pass all retrieved facts to the generator. Without reranking, the most relevant and informative facts may be truncated, leading to incomplete reasoning chains or missing key evidence. This motivates the investigation of reranking and pruning strategies, where both structural relevance and semantic similarity to the query are considered. Such optimization aims to deliver to the generator a context that is not only relevant and coherent, but also compact enough to fit within the model’s constraints, thereby improving factual accuracy and reasoning depth in multi-hop question answering [Es+25].

By empirically comparing subgraph-based retrieval with representative baselines that use unstructured document retrieval, this thesis assesses the trade-offs between structured and unstructured strategies in RAG primarily in terms of answer correctness. Beyond this main metric, we also report practical observations on scalability, retrieval latency, and computational overhead encountered during realistic experimentation.

1.2 Research Questions

The research is guided by the following questions:

1. **RQ1:** To what extent does the quality of the pretrained retriever model—and the resulting extracted subgraphs—affect the overall performance of the RAG pipeline and the quality of its final generated answers?
2. **RQ2:** In subgraph-based RAG, how does the structuring and pruning of retrieved subgraphs, culminating in query-focused summarization, influence answer correctness under a fixed context budget?
3. **RQ3:** Can a method that leverages the public Wikidata knowledge base, combined with a pretrained multi-hop KBQA retriever on the WebQuestionsSP dataset, achieve comparable or superior answer correctness—evaluated against gold-standard answers using an LLM-as-a-judge—when compared to vanilla RAG and more advanced RAG pipelines?

1.3 Contribution of the Thesis

A central contribution is the adaptation and integration of the Subgraph Retrieval Enhanced Model (SREM) [Zha+22], into a RAG-style question answering framework. The pipeline combines question–entity linking with a trained subgraph retriever, and introduces lightweight pre-reranking, pruning, and query-focused summarization to yield compact and relevant graph-derived contexts for a large language model. The framework supports multi-hop questions through a 2-hop retrieval setting, where extracted triples are merged into head-centric subgraphs and subsequently transformed into structured textual summaries. This approach emphasizes both retrieval quality and context efficiency, reducing noise while preserving essential evidence for answer generation.

From an engineering perspective, the thesis contributes a dual-format adaptation of the WebQSP-WD dataset [SG18], a Wikidata-aligned version of the original WebQSP dataset [Yih+16]. This involved mapping entities from questions and answers to Wikidata identifiers, aligning them with relevant Wikipedia articles, and curating a set of 792 Wikipedia documents to serve as the unstructured counterpart to the structured Wikidata triples. This resource enables controlled, head-to-head comparisons between structured subgraph-based retrieval and unstructured document retrieval.

The evaluation proceeds in two stages. First, we conduct intermediate assessments of the retriever by training with varying hyperparameters and selecting a small set of Pareto-optimal models that balance retrieval accuracy and efficiency. These models are then integrated into the subgraph-based pipeline to observe their downstream impact. Second, we perform a final comparison against the baseline using the RAGAS framework [Es+25], focusing on answer correctness as the primary metric for end-to-end performance.

All code, datasets, and experiment configurations used in this thesis are openly available at <https://github.com/SaltyRain/subgraphrag>. The experiments are documented, reproducible, and can be executed directly from the provided repository, ensuring transparency and enabling further research building on this work.

This thesis advances the field of Retrieval-Augmented Generation (RAG) by investigating the integration of structured knowledge graph retrieval into modern language generation pipelines. While prior work has predominantly focused on building custom indices—either over unstructured documents or graph-like representations derived from

text—this research explores an index-free alternative: leveraging an existing large-scale knowledge graph (Wikidata) via a pretrained subgraph retriever, without any additional indexing or offline graph construction.

The key contributions of this thesis are as follows:

- Development of a multi-hop graph retrieval pipeline capable of merging retrieved triples into coherent subgraphs for downstream generation.
- Introduction of a lightweight pre-reranking and pruning mechanism to filter noisy subgraphs and preserve compact, query-relevant evidence.
- Proposal of a query-focused summarisation step that transforms structured subgraphs into concise textual contexts suitable for large language models.
- Creation of a benchmark-ready dataset pairing Wikidata entities with thematically matched Wikipedia articles to enable fair structured–unstructured retrieval comparisons.
- Design of an evaluation workflow combining standalone retrieval metrics with direct cross-method comparisons.
- Full open-source release of the system, datasets, and reproducible experiment setup to support future research.
- Empirical demonstration that effective subgraph retrieval can be achieved without building a dedicated index, using only a pretrained retriever over an existing knowledge graph.

1.4 Structure of the Thesis

This thesis adopts a systematic approach to investigate the integration of subgraph retrieval over structured knowledge graphs into Retrieval-Augmented Generation (RAG) pipelines. Following the introductory chapter, Chapter 2 provides the theoretical foundation for this work. It begins with an overview of Large Language Models (LLMs) and their limitations in knowledge-intensive tasks, providing the motivation for Retrieval-Augmented Generation (RAG) as a means to extend LLM capabilities with external knowledge. The section then outlines the core architecture of RAG, describing the naive

RAG pipeline and common extensions. This is followed by a discussion of graph-based RAG, including its main processing stages and how it differs from conventional document retrieval approaches. The chapter then focuses on subgraph retrieval in knowledge graphs, introducing the concept in the context of multi-hop question answering and structured retrieval.

Two dedicated sections expand the background: the first addresses evaluation in RAG, discussing the challenges of assessing both retrieval and generation components, and reviewing relevant metrics and frameworks for measuring answer correctness, context quality, and faithfulness. The second covers optimization in RAG, with a particular emphasis on retrieval reranking. The chapter concludes with a review of related work in RAG, knowledge graph question answering, and retrieval optimization.

Chapter 3 presents the methodology and implementation of the proposed approach, structured around two main contributions. The first part describes the creation of a unified dataset for comparative evaluation. This includes preparing the training set, formatting the test set, and aligning question-answer entities from WebQSP-WD [SG18] with corresponding Wikipedia articles to enable both structured and unstructured retrieval experiments under identical conditions.

The second part details the subgraph-based RAG architecture developed in this thesis. It describes the integration of the Subgraph Retrieval Enhanced Model (SREM) [Zha+22] into a RAG pipeline, including the training procedure, the application of a 2-hop retrieval setting, and the construction of connected subgraphs from retrieved triples via intermediate path assembly. These subgraphs are then converted into natural language context for a large language model. The chapter concludes with the answer generation process, describing how the retrieved and ranked structured context is used to produce final outputs.

Chapter 4 outlines the complete experimental workflow, beginning with the Experimental Setup. This section covers the development environment — including hardware specifications — and the system configuration, which comprises the local SPARQL endpoint, the Ollama-based LLM and embedding services, and the integration of the Subgraph Retrieval Toolkit (SRTK) [She23] built on the Subgraph Retrieval Enhanced Model (SREM) [Zha+22]. It also includes summary statistics on WebQSP-WD Dataset filtering outcomes and Wikipedia articles corpus composition.

The setup further includes the preparation of supervised training data for the retriever, involving 1-hop and 2-hop path extraction, recall-based positive filtering, and negative sampling. Several embedding models and training configurations are explored, with learning curves and summary statistics on the quality of the resulting retrieval data presented.

This chapter further reports three experiments.

Experiment 1 (Retriever selection). We train and probe multiple retriever configurations under controlled settings (fixed corpus and traversal budgets), and select a small Pareto set that balances retrieval accuracy and efficiency. Diagnostics include recall over answer entities, subgraph size ($\#nodes/\#edges$), entity coverage, and basic timing.

Experiment 2 (End-to-end RAG with subgraphs). The selected retrievers are integrated into our subgraph-based RAG pipeline, which applies pre-reranking, pruning, and query-focused summarization.

Experiment 3 (Baselines). Finally, we compare the proposed pipeline against two document-based RAG baselines—(i) a naive dense-retrieval setup using unstructured documents aligned with the question entities and (ii) LightRAG [Guo+25]—implemented on the same corpus and under identical conditions.

Chapter 5 provides a critical discussion of the experimental results in the context of prior research. It analyses the strengths and limitations of structured retrieval with a fine-tuned subgraph retriever, highlighting the impact of multi-hop traversal, beam width, and reranking strategies on retrieval and generation quality. The discussion considers the practical implications of an index-free setup based on an existing large-scale knowledge graph, examines the scalability and latency trade-offs compared to document-based baselines, and evaluates the generalizability of the approach to other knowledge graphs and knowledge-intensive tasks. Methodological design choices are reviewed, including dataset preparation, retriever training configuration, and evaluation methodology, with suggestions for further improvements such as deeper multi-hop reasoning, adaptive retrieval parameters, and hybrid structured–unstructured pipelines.

Chapter 6 concludes the thesis by summarizing the core contributions, key findings, and implications of this work. It reinforces the potential of structured graph-based retrieval as a scalable and effective alternative to conventional document-based approaches in RAG pipelines, especially for complex, multi-hop question answering. Finally, it outlines open research directions, including broader evaluations across domains, integration with larger and more diverse knowledge graphs, and the development of hybrid systems that combine the precision of structured retrieval with the flexibility of unstructured sources.

2 Background

This chapter provides the theoretical foundations for this thesis and contextualizes the proposed approach within the broader landscape of retrieval-augmented generation. It begins with an overview of Retrieval-Augmented Generation (RAG), introducing how large language models (LLMs) can be enhanced through the integration of external retrieval mechanisms (Section 2.1). The chapter then presents GraphRAG as a variant of RAG that operates over knowledge graphs rather than unstructured text, outlining its core components and the advantages of graph-based reasoning (Section 2.2). The final section focuses on subgraph retrieval as a key retrieval granularity in GraphRAG systems, highlighting its role in enabling multi-hop reasoning and introducing the trainable subgraph retriever used in this work, which dynamically queries Wikidata at runtime (Section 2.2.2). A more detailed presentation of related methods and retrieval strategies is provided in Section 2.5.

2.1 Retrieval-Augmented Generation: Concept and Architecture

Large language models (LLMs) have demonstrated remarkable capabilities across a range of natural language understanding and generation tasks [Min+25; Bro+20]. However, their reasoning is fundamentally constrained by the static nature of their parametric knowledge—information encoded during pretraining. As a result, they struggle to respond accurately to queries requiring up-to-date, domain-specific, or factual knowledge that lies beyond their training data [Kan+23]. In addition, reliance on fixed training data introduces a tendency to produce fluent yet factually incorrect statements, commonly referred to as hallucinations, which can undermine the reliability of LLM-generated responses [Ji+23; Hua+25]. This has motivated the development of architectures that can dynamically incorporate external sources of information at inference time [Lew+21].

2 Background

Retrieval-Augmented Generation (RAG) is one such architecture that combines large language models with non-parametric retrieval to improve factual accuracy and adaptability in knowledge-intensive tasks. A typical RAG model consists of two components: a retriever and a generator. The retriever, often based on dense or sparse retrieval methods (e.g., Dense Passage Retrieval (DPR) [Kar+20], Best Matching 25 (BM25) [RZ09]), searches an external knowledge base (such as Wikipedia or a vector store) to retrieve the top- k most relevant documents. These documents are then concatenated with the original query and passed to the generator (e.g., BART [Lew+19] or T5 [Raf+23]), which produces the final answer.

Formally, the generation model treats the retrieved documents as latent variables and defines the output distribution as a marginal over the top- k retrieved contexts. Given an input x and retrieved passages $z_1, \dots, z_k$, the model generates an output y by computing:

$$p(y|x) = \sum_{i=1}^k p(z_i|x) \cdot p(y|x, z_i).$$

Two architectural variants have been explored in the literature: *RAG-Sequence*, where a single retrieved document is used to generate the entire output, and *RAG-Token*, which allows each token to be conditioned on a different retrieved passage [Lew+21].

This probabilistic formulation enables end-to-end training of both retriever and generator while accounting for uncertainty in retrieval. Additionally, the architecture supports a separation of memory and computation: external information can be updated independently of the model parameters, allowing the system to remain current without retraining the underlying language model.

While the original RAG formulation supports end-to-end probabilistic reasoning and joint training of retriever and generator [Lew+21], many real-world systems adopt a simplified implementation pattern that retains the core retrieve-and-generate mechanism without full probabilistic modeling or retriever fine-tuning. This practical approach, often referred to as Vanilla RAG, serves as the foundation for most current RAG applications and forms the baseline upon which more advanced systems—such as graph-based retrieval architectures—are built.

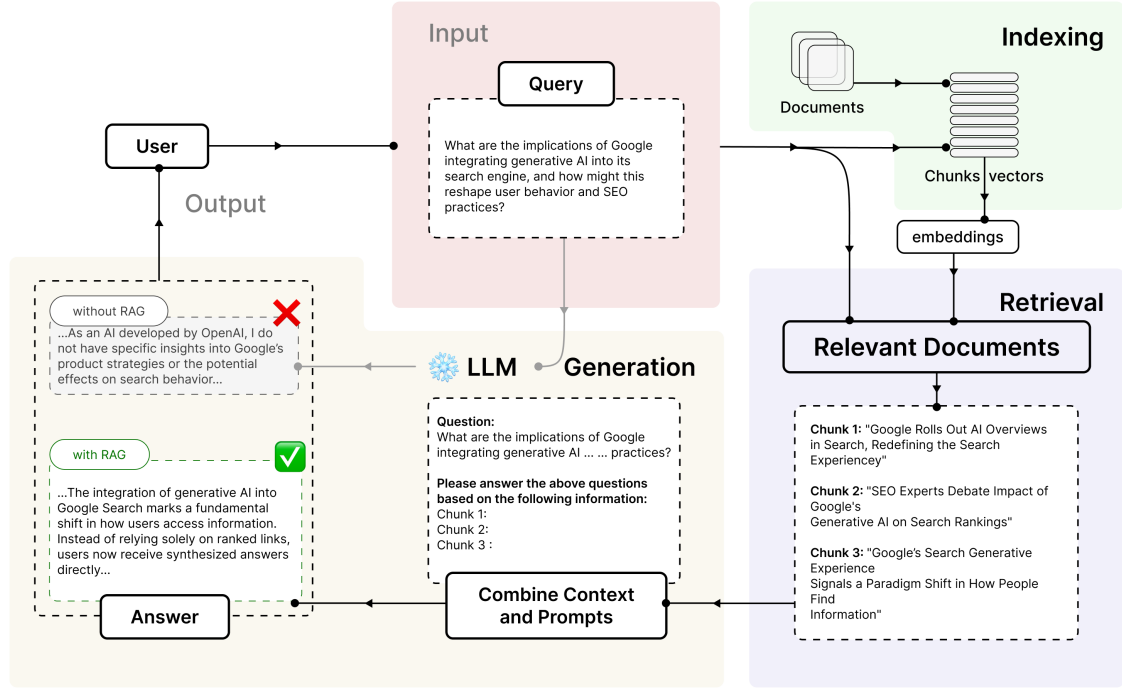


Figure 2.1: A typical RAG pipeline for question answering with three stages: (1) Indexing — segmenting and embedding documents into a vector store; (2) Retrieval — selecting top- k relevant chunks by similarity; (3) Generation — producing an answer using the query and retrieved context.

2.1.1 Naive RAG Pipeline

Building on this simplified design, Naive RAG implementations typically follow a retrieve-read pipeline [Gao+24], as illustrated in Figure 2.1. Systems begin with an indexing stage, where raw documents in formats such as PDF, HTML, or Markdown are cleaned and converted into plain text. To accommodate the input length limitations of language models, the text is segmented into smaller chunks, which are then embedded into vector representations and stored in a vector database.

During inference, the input query is encoded using the same embedding model and matched against the indexed vectors via similarity search. The top- k most relevant chunks are retrieved and injected into the prompt passed to the language model. The final response is then generated, either relying solely on the retrieved content or combining it with the model’s parametric knowledge. This paradigm, often referred to as “Naive RAG,” has become a foundational implementation pattern for real-world RAG

systems.

2.1.2 Advanced RAG Approaches

Since their introduction, RAG architectures have demonstrated improved factual accuracy and adaptability across open-domain question answering, fact verification, and knowledge-intensive tasks [Lew+21; IG20]. By grounding their responses in retrieved evidence, these systems mitigate hallucinations and improve interpretability, as each answer can be traced to specific sources [Ji+23; Shu+21].

Several extensions of the original RAG framework have been proposed to improve retrieval accuracy, reasoning capability, and factual grounding [Gao+24]. These efforts have led to what is often referred to as *Advanced RAG*—a family of approaches that aim to overcome the limitations of the naive retrieve-and-generate paradigm.

Advanced RAG systems enhance retrieval in both the pre-retrieval and post-retrieval phases. In the pre-retrieval stage, the focus is on optimizing document indexing and query formulation. This includes strategies such as fine-grained text segmentation, sliding window indexing, metadata enrichment, and query rewriting or expansion [Ma+23; Pen+24b; Gao+22]. These improvements aim to ensure that relevant information is captured and that queries are better aligned with the structure of the indexed content.

In the post-retrieval phase, the goal is to integrate retrieved content more effectively. This involves reranking retrieved passages to prioritize the most relevant segments [Bla23] and context compression techniques to reduce input size without losing critical information. These methods mitigate problems such as information overload and the “lost in the middle” phenomenon by selecting and emphasizing salient information before generation.

Despite these advances, RAG systems continue to face key limitations. Their effectiveness remains highly sensitive to retrieval quality, and they struggle with tasks requiring multi-hop reasoning or global synthesis across semantically distant passages [Min+23; Ji+23]. Moreover, the unstructured nature of retrieved text can hinder precise reasoning, especially when evidence must be integrated in a logically consistent way.

To address these challenges, recent research has turned toward incorporating **structured knowledge representations**, such as **knowledge graphs**, into the RAG pipeline. By

explicitly encoding entity relationships and leveraging graph traversal during retrieval, these systems—referred to as **GraphRAG**—aim to provide more focused and logically coherent context for generation. This direction is explored in the following section.

2.2 Graph-Based Retrieval-Augmented Generation

Graph-Based Retrieval-Augmented Generation (GraphRAG) [Pen+24a] is an emerging paradigm that enhances classical RAG systems by retrieving structured relational knowledge, such as nodes, triples, or subgraphs, from preconstructed knowledge graphs (KGs). Unlike unstructured text retrieval, this approach captures explicit semantic relationships between entities, enabling more accurate multi-hop reasoning and more compact input representations [Pen+24a; Edg+25]. By grounding generation in a graph-based context, GraphRAG improves factual consistency, interpretability, and robustness to irrelevant or redundant information, especially in tasks like entity-centric summarization and complex question answering.

2.2.1 GraphRAG Pipeline

The GraphRAG workflow can be divided into three main stages: graph-based indexing, graph-guided retrieval, and graph-enhanced generation [Pen+24a].

Graph-based indexing involves identifying or constructing a knowledge graph G that aligns with the downstream task. This includes defining node and edge attributes, connecting entities via relations, and organizing the graph for efficient traversal and access. The indexing stage determines the granularity of the retrieval and significantly affects performance and relevance in later stages. Graph databases used in existing GraphRAG systems are typically derived from various sources: public knowledge graphs such as Wikidata, Freebase, or DBpedia [Aue+07; Bol+08; VK14], curated graph datasets [Mor+20], or proprietary resources constructed from raw textual or domain-specific data [Edg+25; Guo+25; Luo+25; Gut+25]. These approaches often require extensive preprocessing, entity linking, and relation extraction to build task-specific graph indices.

A notable example is LightRAG [Guo+25], which segments documents, extracts entities and relations via LLMs, and enriches them through a profiling step that generates key-value pairs for retrieval. Identical entities and relations from different segments are merged to produce a deduplicated, enriched graph. Retrieval operates over textualized keys using sparse methods such as BM25 or SPLADE, enabling fast access without explicit graph traversal. While this preconstructed index supports multi-hop reasoning, it depends heavily on extraction accuracy and may retain noise from the original documents.

In contrast to prior work, which typically relies on preprocessed graph indices, the method proposed in this thesis adopts a dynamic retrieval strategy: rather than constructing a custom index, we query Wikidata directly at runtime to retrieve subgraphs relevant to the input query.

Graph-guided retrieval aims to extract relevant information given a user query q which is written in natural language. At this stage, we aim to extract the most relevant elements - entities, triplets, paths, subgraphs - from the knowledge graphs.

Graph-enhanced generation phase involves creating relevant and meaningful responses based on the retrieved information. At this stage, the generator combines the query, the retrieved graph elements, and an optional input to generate a response.

2.2.2 Subgraph Retrieval in Knowledge Graphs

A critical component of GraphRAG systems is the ability to retrieve a relevant subgraph from a large knowledge graph (KG) in response to a user query. Unlike traditional RAG systems that return unstructured textual passages, knowledge-graph-based retrieval operates at varying levels of granularity — including nodes, triplets, paths, and subgraphs — depending on the task and retrieval strategy [Pen+24a].

Among these, **subgraph retrieval** is particularly well-suited for complex question answering, as it captures both local and multi-hop relational contexts. A subgraph may consist of semantically connected entities and relations, offering richer interpretability and support for structured reasoning compared to isolated nodes or flat triplets.

Retrieving subgraphs at scale, however, remains a challenging problem. Heuristic methods based on k -hop expansion or path enumeration often fail to balance completeness

and noise, leading to either insufficient or overly large candidate subgraphs. To address this, recent work has proposed trainable subgraph retrievers that operate independently of downstream reasoning modules. In particular, the method proposed by Zhang et al. [Zha+22] trains a retriever to identify relevant reasoning paths and merges them into a unified subgraph, achieving strong performance on multi-hop KBQA tasks.

Inspired by this approach, we adopt a similar retrieval strategy: our pipeline employs a trained subgraph retriever to select semantically relevant paths from Wikidata based on the input question. These paths are then merged into a coherent subgraph of interconnected triplets, which serves as the structured input to the generation module. This setup allows for dynamic and query-specific retrieval at runtime, improving semantic coverage while maintaining efficiency.

2.3 Evaluation in Retrieval-Augmented Generation

Evaluating Retrieval-Augmented Generation (RAG) systems requires a dual focus: assessing the final answer quality and verifying the suitability of the retrieved context. Unlike purely generative models, RAG architectures explicitly separate retrieval and generation, meaning that evaluation must capture both stages.

Traditionally, answer quality is measured using reference-based metrics such as Exact Match (EM) and token-level F1, which reward precise lexical matches with gold-standard answers. However, these metrics can undervalue semantically correct but lexically different outputs, a limitation especially relevant in open-ended or multi-hop question answering. To address this, recent approaches employ Large Language Models as evaluators (“LLM-as-a-judge”), which can assess responses for semantic correctness, coherence, and informativeness without relying solely on surface-level overlap [Zhe+23].

Context evaluation is equally important in RAG, as even a factually correct answer can be the result of hallucination rather than grounding in retrieved evidence. Metrics such as context precision and recall quantify how much of the retrieved material is relevant and how much of the relevant material has been retrieved, respectively. Faithfulness measures the extent to which the generated answer is directly supported by the provided context, penalizing unsupported or contradictory statements [Es+25]. This dimension is particularly critical in knowledge-intensive tasks, where the credibility of the output

depends on its verifiable grounding. A concise summary of these evaluation dimensions is provided in Table 2.1.

Table 2.1: Summary of evaluation dimensions in RAG

Metric	What it measures	Purpose in RAG
Exact Match (EM)	Binary correctness of predicted answer vs. gold reference	Strict lexical match check
F1 Score	Token-level overlap between prediction and gold reference	Balances precision and recall in lexical matching
LLM-as-a-judge	Semantic correctness, coherence, informativeness	Captures meaning beyond lexical similarity
Context Precision	Fraction of retrieved context that is relevant	Minimizes noise in retrieval
Context Recall	Fraction of relevant context that was retrieved	Ensures coverage of necessary facts
Faithfulness	Degree to which answer is supported by retrieved evidence	Prevents hallucinations and unsupported claims

In this thesis, we initially explored both reference-based and context-oriented evaluation. However, direct context comparison proved unreliable, since our framework generates query-focused summaries while document-based baselines retrieve raw chunks, making the two outputs not directly comparable. We therefore focus our final evaluation exclusively on answer correctness, which provides a fair and consistent basis for assessing system performance.

2.4 Optimization in Retrieval-Augmented Generation

The effectiveness of RAG pipelines depends heavily on how well the retrieval stage delivers relevant and useful context to the generator. Optimization can be applied before, during, and after retrieval.

Pre-retrieval Optimization aims to maximize the chance of retrieving relevant material by improving query representation. This can involve semantic query reformulation, entity linking to align queries with knowledge graph entities, or hybrid retrieval strategies that combine sparse lexical matching (e.g., BM25) with dense semantic embeddings [RZ09; Kar+20].

To further prioritize candidate subgraphs before traversal, lightweight scoring signals can be used. These include vector-space similarity between bag-of-words representations of the query and subgraph stubs [MRS08; SB88], Jaccard-style overlap between recognized entities and subgraph nodes as a proxy for seed recall, and structural heuristics that combine relation diversity with edge density [New10]. Normalizing and combining such measures provides an efficient way to retain subgraphs that are both topically aligned and structurally informative.

Retrieval-level optimization refines the ranking of candidate results. Such approaches have been shown to significantly improve retrieval precision while keeping latency manageable [Bla23]. In graph-based settings, reranking can also exploit topological features such as path connectivity and node centrality [Zha+22].

Post-retrieval optimization focuses on how the retrieved content is integrated into the LLM input. Context compression methods select only the most salient fragments to fit within model token limits [Lew+21], while selective grounding discards irrelevant or low-confidence passages to reduce noise. In GraphRAG, the construction of semantically coherent subgraphs before generation effectively serves as a form of post-retrieval optimization, ensuring that only interconnected, relevant triples are passed to the model.

The approach presented in this thesis combines all three stages of optimization: semantic enrichment and entity linking in pre-retrieval, path-based reranking during retrieval, and structured subgraph assembly with context compression after retrieval. This design aims to achieve a balance between recall, precision, and faithfulness, particularly in multi-hop question answering scenarios.

2.5 Related Work

This chapter surveys prior research that forms the conceptual and technical basis for this thesis. It traces the evolution of Retrieval-Augmented Generation (RAG) from early

2 Background

Table 2.2: Summary of related retrieval-augmented generation methods

Work / Year	Data	Retrieval	Core Technique	Pros	Limitations
RAG (Lewis et al., 2020)	Unstructured	Dense (DPR)	Neural retriever + seq2seq	End-to-end; strong factual QA	No structured KG use
GraphRAG (Edge et al., 2024)	Mixed	Graph-based	Entity graph; community summaries; subgraph retrieval	Multi-doc reasoning; better aggregation	Graph build/-maintenance overhead
LightRAG (Guo et al., 2024)	Triples	Sparse	Triples-as-text indexing; dual-use	Efficient; low latency; simple infra	Limited deep relational reasoning
HippoRAG (Gutiérrez et al., 2024)	KG	Traversal	Personalized PageRank for node selection	Topological relevance; scalable traversal	Sensitive to KG noise/incompleteness
GNN-RAG (Mavromatis et al., 2024)	KG	GNN-based	Learned node/sub-graph embeddings	Captures implicit links; robust to gaps	Requires re-training; weak transfer
GFM-RAG (Luo et al., 2025)	KG	Pretrained GFM	Query-dependent GNN pre-trained on 60 KGs	Zero-shot generalization across domains	High pretraining cost; heavy footprint

document-based retrieval to recent advances in graph-based architectures and subgraph retrieval methods, with a focus on systems that integrate structured knowledge into the retrieval-generation pipeline.

The RAG architecture was first introduced by Lewis et al. (2020) in “*Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*” [Lew+21]. In their framework, a dense retriever (DPR) selects top- k relevant documents from a large corpus, and a sequence-to-sequence generator (e.g., BART) produces the answer conditioned on both the query and retrieved documents. Two variants were proposed: **RAG-Sequence**, where the same document is used for the entire output sequence, and **RAG-Token**, where different tokens can attend to different documents. This approach outperformed purely parametric models and classic retrieve-then-read pipelines, but relied exclusively on unstructured document retrieval without leveraging structured knowledge graphs (KGs).

The move toward **graph-based retrieval** emerged as a way to exploit the explicit structure of entities and relations in KGs. Edge et al. (2024) introduced **GraphRAG** [Edg+25], which builds a graph index over the source corpus, retrieves a subgraph relevant to the query, generates community-level summaries, and composes them into a global answer. This enables multi-document reasoning and richer contextualization

compared to flat text retrieval.

LightRAG [Guo+25] extends this paradigm with support for incremental updates, allowing new documents to be integrated without full reindexing. In this thesis, it is used as a baseline, providing a low-latency, index-based alternative to our subgraph-based method, which performs explicit multi-hop traversal before generation.

Beyond RAG-specific frameworks, subgraph retrieval itself is a well-established research area within knowledge graph querying and information extraction. Common approaches include breadth-first search (BFS) for exhaustive neighborhood exploration, which is simple but can be inefficient for large graphs; **Personalized PageRank (PPR)**, a random-walk-based relevance measure widely used in graph search and exploration tasks [JW03; Par+19]; and **embedding-based matching**, which embeds entities and relations into dense vector spaces to enable semantic-related neighborhood expansion and support inference and link prediction tasks [Cho+21]. Hybrid methods, such as **GraftNet**, combine symbolic traversal (e.g., PPR) with neural models (e.g., graph convolutional networks) to refine retrieval for QA over knowledge graphs [Sun+18]. The subgraph retrieval module proposed in this thesis builds upon these paradigms, adapting them to the demands of dynamic, graph-based RAG systems with an emphasis on efficiency and interpretability.

Subgraph extraction techniques also evolved. **HippoRAG** [Gut+25] uses personalized PageRank over KGs to select contextually relevant nodes based on topological relevance. This is effective when the KG is reliable but can degrade if the graph is noisy or incomplete.

A separate line of research incorporates **Graph Neural Networks (GNNs)** for retrieval [MK24; He+24]. GNN-based retrievers learn node and subgraph embeddings that capture both explicit and implicit relationships, enabling reasoning even on partially missing graphs. However, they typically require dataset-specific training and may not generalize well across domains.

Hybrid symbolic–neural methods attempt to combine robustness with adaptability. Galkin et al. (2024) [Gal+24] use a two-stage pipeline: candidate subgraphs are extracted heuristically, then refined via a GNN.

The most recent trend is toward **foundation models for graph retrieval**. **GFM-RAG** [Luo+25] pretrains a query-dependent GNN retriever on 60 KGs and 700k doc-

2 Background

uments, enabling zero-shot generalization to unseen domains. This is the first large-scale graph foundation retriever, achieving state-of-the-art performance on multi-hop and domain-specific QA without fine-tuning.

A concise comparison of the reviewed RAG frameworks, highlighting their retrieval strategies, advantages, and limitations, is provided in Table 2.2.

3 Methods

This chapter describes the methodology used in this thesis, consisting of two main components. First, we introduce a custom dataset construction process designed to support comparative evaluation across both graph-based and text-based RAG systems. Using the WebQSP-WD dataset [SG18] as a foundation, we extract a corresponding corpus of Wikipedia articles that are relevant to the test questions. This corpus serves as the textual knowledge base for baselines such as LightRAG [Guo+25] and naive RAG [Lew+21], enabling fair comparisons between different retrieval strategies within a unified experimental framework.

In addition to extracting a textual corpus, we apply several preprocessing steps to the original WebQSP-WD dataset. These include data cleaning, structural normalization, and the removal of noisy or inconsistent samples. Furthermore, we perform entity linking for the training partition, aligning question-answer pairs with corresponding Wikidata identifiers. This enriched training set is subsequently used in the supervised training of the subgraph retriever described in later sections.

Second, we present a subgraph-based RAG pipeline, referred to as *SubgraphRAG*, which integrates a trainable subgraph retriever inspired by the architecture proposed in [Zha+22] into a full end-to-end RAG system. This pipeline dynamically retrieves relevant subgraphs from Wikidata [VK14] based on the input query and uses them as structured context for answer generation.

3.1 Unified Dataset for Comparative Evaluation

This work builds upon the WebQSP-WD dataset (v1.0, 2018) [SG18], an alignment of the original WebQSP dataset [Yih+16] to the Wikidata knowledge graph. The dataset

contains natural language questions paired with corresponding answer entities represented as Wikidata Q-IDs. In this thesis, we use both the training and test partitions to support different stages of model development and evaluation.

To better suit the needs of our pipeline, we reformulate the dataset structure through field remapping and normalization. In particular, the original training data lacks explicit annotations of question-linked entities, which are required for supervised subgraph retriever training. To address this, we introduce a custom entity linking pipeline that identifies and maps question mentions to Wikidata entities. This pipeline leverages both Wikipedia-based heuristics and external APIs to ensure high-quality alignments. The resulting linked entities serve as input nodes for subgraph retrieval during training.

For the test partition, pre-linked question entities are available and used directly to guide subgraph extraction and answer evaluation. These entities also form the basis for constructing an auxiliary textual corpus: we collect corresponding Wikipedia articles for all test-time entities, enabling evaluation of both structured (graph-based) and unstructured (text-based) retrieval strategies within a unified framework.

Importantly, this work does not rely on any of the auto-generated graph annotations included in the original dataset distribution (commonly referred to as *silvergraphs*). Instead, we implement a fully independent subgraph retrieval pipeline that queries external knowledge sources directly, allowing for reproducible and extensible experimentation.

The resulting processed dataset—comprising clean train/test splits, entity-linked annotations, and a Wikipedia-derived textual corpus—constitutes a reusable resource for future work on graph-augmented question answering over Wikidata.

3.1.1 Train Set Preparation

Table 3.1 summarizes the field normalization process applied to the original **train** partition of the dataset, aligning its structure with the input requirements of the retriever training pipeline. Following this preprocessing step, each instance consists of a question identifier, the natural language question text, and a list of associated answer entities represented by Wikidata Q-IDs. An additional field containing question-linked entities is appended via entity linking, as described in the following section and illustrated in Figure 3.1.

Original Field	Mapped Field
questionid	id
utterance	question
answers	answer_entities
answers_str	(omitted)
–	question_entities

Table 3.1: Field mapping from original train dataset partition to cleaned format.

Because the training set lacks pre-annotated question entities, entity linking is conducted using the TAGME [FS10] API¹. Each question is issued as a single API request, with no filtering applied at retrieval time. All annotations returned by the API are retained and later filtered locally as part of a dedicated postprocessing stage.

To maximize recall and minimize the risk of excluding relevant entities, *lenient thresholds* are applied: link probability ≥ 0.004 and semantic relatedness $\rho \geq 0.004$. This conservative strategy removes only the most irrelevant mentions, while preserving the majority of semantically meaningful entities, albeit with some tolerated noise.

After filtering the TAGME annotations, the extracted Wikipedia titles are mapped to their corresponding Wikidata entity identifiers using the **wikimapper** tool [Kli18]. This step produces a list of question entities, which is stored in an intermediate file and later used to populate the **question_entities** field required by the subgraph retrieval framework **srtk** [She23].

Mapping to Wikidata IDs and Final Formatting

The mapping is performed using an offline SQLite index provided by **wikimapper**, based on the April 20, 2019 Wikipedia dump (**index_enwiki-20190420.db**). The resulting Q-IDs serve as training-time question entities.

Each question is then combined with its answer entities (from the cleaned training set) to form the input structure expected by the retriever: question ID, question text, a list of question entities, and a list of answer entities.

¹<https://sobigdata.d4science.org/web/tagme/tagme-help>

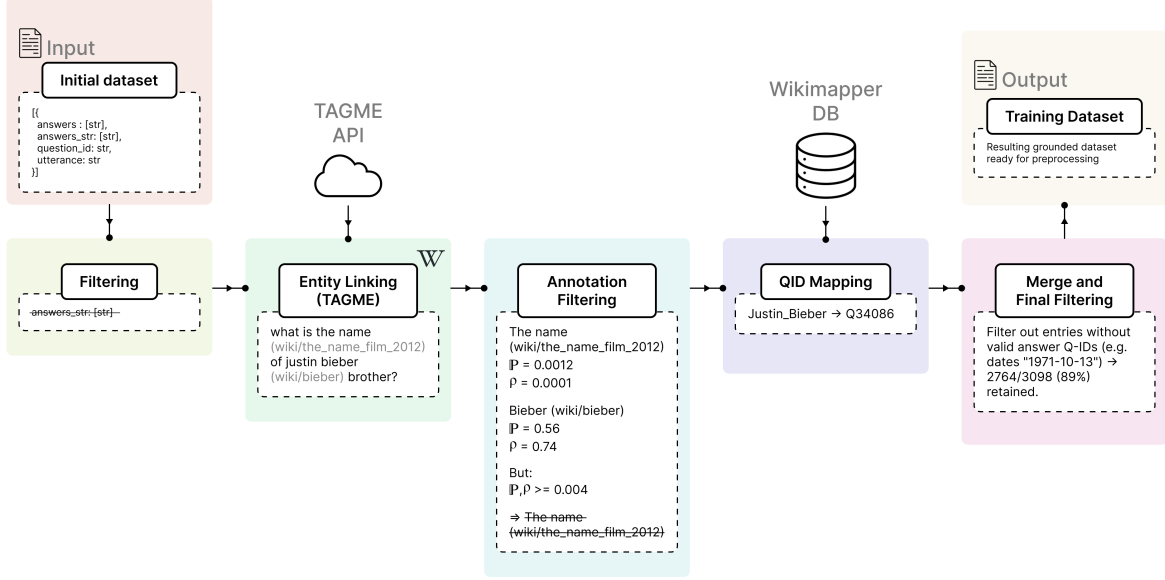


Figure 3.1: Workflow for preparing grounded training set for the subgraph retriever

In this step, invalid or incomplete entries are discarded — for example, questions lacking valid answer Q-IDs or containing placeholders such as date values. Only entries with at least one valid answer entity (i.e., a Wikidata ID starting with “Q”) are retained.

After filtering, the final training set contains 2,764 examples, representing approximately 89% of the original 3,098 training questions.

3.1.2 Test Set Formatting

The original test set contains natural language questions, answer entities, and a nested `entities` field with precomputed entity linkings. To enable consistent inference across different retriever pipelines, we convert this data into a simplified JSONL format aligned with the structure used during training.

Table 3.2 summarizes the field mapping applied to each entry in the test set.

This transformation flattens the nested structure and extracts Wikidata Q-IDs for each question, resulting in a clean representation containing only the necessary fields: question ID, text, linked entities, answer IDs, and answer strings. The formatted test set is used as input to all downstream evaluation components. Although string answers are not

Original Field	Mapped Field
questionid	id
utterance	question
answers	answer_entities
answers_str	answers_str
entities[0].linkings[*][0]	question_entities

Table 3.2: Field mapping from original test dataset partition to evaluation-ready format.

required for evaluating the retriever’s performance, they are retained for subsequent evaluation of final LLM-generated answers within the RAG pipeline.

3.1.3 Wikipedia Article Extraction for Baselines

This step operates on the preprocessed test set described in Section 3.1.2, which contains a list of Wikidata entity identifiers (`question_entities`) for each question. We collect the union of all unique Q-IDs across the test set, resulting in 681 distinct entities.

For each Q-ID, we use the `wikimapper` tool to obtain one or more candidate Wikipedia titles. These raw titles are then expanded using simple normalization rules: we add lowercase variants, replace underscores with spaces, and include all such combinations. This results in a robust set of possible title variants for each entity, improving matching coverage.

In the first stage, we search for these titles within the static English Wikipedia snapshot provided by Hugging Face (`wikimedia/wikipedia`, November 2023)². This dataset contains over 6.4 million articles and is indexed by title. Using our expanded titles, we extract 615 articles in plain text format from this offline corpus.

For Q-IDs with no matching titles in the static snapshot (177 cases), we fall back to querying the official Wikipedia API. We iterate through each candidate title and store the first successfully retrieved page. This dynamic stage successfully recovers all remaining articles, resulting in 792 documents in total. Some entities yield multiple semantically related or redirect-based articles, which are intentionally preserved.

²<https://huggingface.co/datasets/wikimedia/wikipedia/viewer/20231101.en>

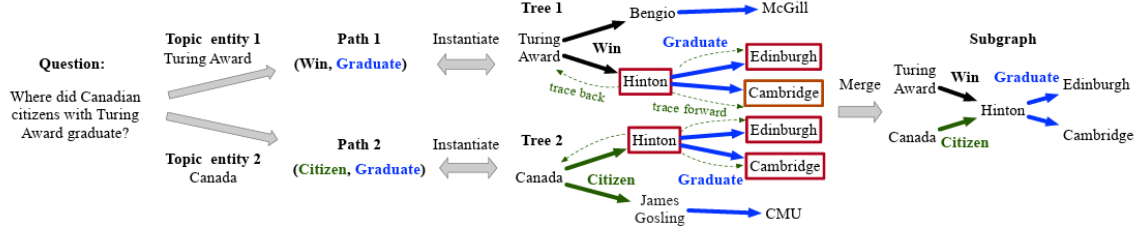


Figure 3.2: Illustration of the subgraph retrieving process. Taken from Zhang et al. (2022) [Zha+22].

The final corpus combines articles retrieved from both the Hugging Face dataset and the Wikipedia API. This hybrid approach ensures high coverage while maintaining reproducibility for offline use.

3.2 Subgraph-Based RAG Architecture

3.2.1 Subgraph Retriever Training

The subgraph retriever is a neural scoring model designed to guide symbolic graph traversal during multi-hop question answering. At each step of the traversal, it ranks all candidate relations originating from the current node, based on a natural language question and the sequence of previously followed relations. The highest-scoring relations are then selected to extend the traversal path, gradually constructing a subgraph that contains relevant knowledge for answering the question. This process is illustrated in Figure 3.2, which shows how multiple relation paths are expanded from different topic entities and merged into a unified subgraph.

This method builds upon the Subgraph Retriever (SR) framework introduced by Zhang et al. [Zha+22] and uses Subgraph Retrieval Toolkit (SRTK) [She23], which decouples subgraph retrieval from downstream reasoning. By training the retriever independently, SR enables plug-and-play integration into knowledge-based QA systems, including those based on retrieval-augmented generation (RAG).

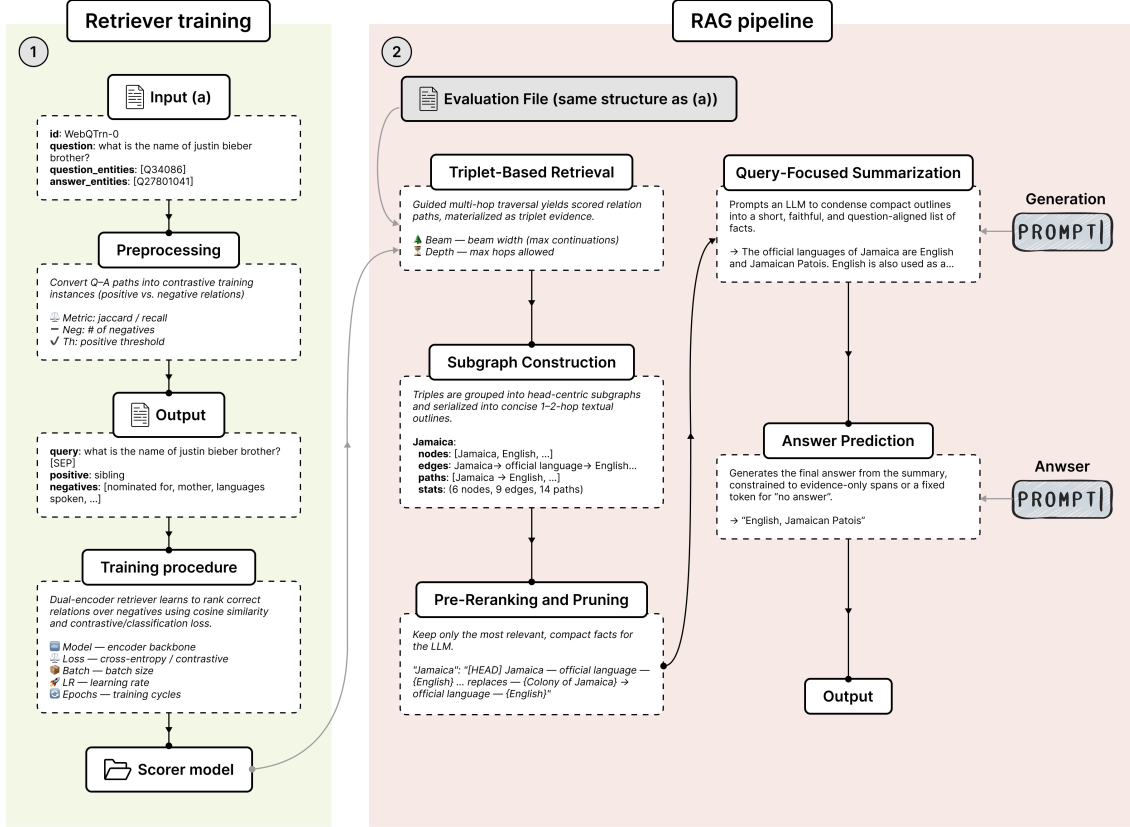


Figure 3.3: Overview of the full SubgraphRAG pipeline, including retriever training and the main pipeline stages.

Train Set Preprocessing

To enable supervised training of the retriever, a dataset of graph traversal examples is constructed to simulate multi-hop reasoning over a knowledge graph. The preprocessing strategy follows a *weakly supervised learning paradigm* in which meaningful relation paths between question and answer entities are identified and converted into contrastive training instances. The approach generates positive examples from semantically relevant paths and augments them with sampled negative relations at each traversal step. This allows the retriever to learn how to select contextually appropriate relations based on both the question and the traversal history.

The preprocessing pipeline begins with the identification of 1-hop and 2-hop relation paths connecting question and answer entities. These paths are extracted from the knowledge graph (KG) using constrained search strategies designed to capture semanti-

cally relevant reasoning chains without exhaustive exploration. Each discovered path is then evaluated using a scoring metric to assess its alignment with the original natural language question. Two scoring functions are considered: token-level Jaccard similarity between the question and the relation labels, and recall over the answer entity set. Based on the selected metric, a threshold is applied to retain only the most informative paths.

To enable contrastive training, negative examples are generated by sampling relations that do not contribute to reaching the answer entity. For each step along a positive path, a fixed number of such negative relations are included to support learning of discriminative decision boundaries

The relation identifiers used in the graph (e.g., property IDs) are normalized to human-readable textual labels, enabling compatibility with natural language retrievers. Each traversal decision is then transformed into a training instance consisting of a query a query consisting of the original question and the sequence of previously selected relations and a set of candidate relations comprising one positive and multiple negatives.

The resulting dataset approximates the decision-making process of multi-hop traversal in a weakly supervised setting, providing the subgraph retriever with supervision for ranking relation candidates at each step based on both the current graph context and the natural language input. An example entry from the WebQSP-WD dataset is shown in Listing 3.1.

```
{
  "id": "WebQTrn-3",
  "question": "what country is the grand bahama island in?",
  "question_entities": ["Q17036433", "Q866345"],
  "answer_entities": ["Q778"]
}
```

Listing 3.1: Example entry from the WebQSP-WD dataset

The preprocessing pipeline produces a sequence of training instances. Each instance contains the current query context — consisting of the question and, optionally, the path traversed so far — along with the correct relation to follow (positive) and a set of distractor relations (negatives).

An example of such generated training data is shown in Listing 3.2.

```

{"query": "what country is the grand bahama island in? [SEP] ",
 "positive": "country",
 "negatives": ["location", "genre", "director", ...]}

{"query": "what country is the grand bahama island in? [SEP] country",
 "positive": "END OF HOP",
 "negatives": ["capital", "continent", "currency", ...]}

```

Listing 3.2: Sample training instances generated during preprocessing

This representation teaches the retriever to iteratively select semantically appropriate relations that lead from the question entities to the answer entities, while ignoring plausible but incorrect alternatives.

Training Procedure

The retriever model follows a dual-encoder architecture in which both the input query and each candidate relation are independently embedded into a shared vector space. The query encodes the natural language question together with the previously traversed relation path, while candidate relations are represented as standalone textual sequences. Semantic similarity between the query and relation embeddings is computed using cosine similarity, enabling the model to score and rank candidate relations based on their alignment with the query context.

The model is trained using a supervised objective that promotes higher similarity scores for correct relations while minimizing similarity to incorrect alternatives. Two loss formulations are considered: a classification-based objective that selects the correct relation from a set of candidates, and a contrastive loss that directly optimizes pairwise distances in the embedding space. The final embedding representation for each input can be computed via different pooling strategies, such as mean pooling or using a special classification token.

The training is performed using a standard neural optimization framework with mini-batch updates, and includes configurable parameters such as batch size, learning rate, number of training epochs, and the number of negatives per instance. The goal of training is to enable the model to distinguish semantically relevant relations from distractors in a multi-hop traversal setting.

The retriever implementation used in this work builds upon an existing open-source training framework. The specific architectural choices, pretrained components, and parameter settings are detailed in Section 4.3. Multiple configurations were explored during development, and their effect on final performance is discussed in Chapters 4 and 5.

3.2.2 Triplet-Based Retrieval

The retrieval stage produces, for each query, a set of triplets (s, p, o) from the knowledge graph (KG), serving as the symbolic evidence for subsequent subgraph construction. We employ the Subgraph Retrieval Toolkit (SRTK) [She23], which performs entity-centric traversal guided by a learned relation-relevance model.

Defining Inputs and Anchors. The procedure consumes the natural-language question together with pre-annotated `question_entities` (seed entities) prepared during dataset processing (see Section 3.1). No entity linking is performed at this stage.

Performing Beam Search over Relation Paths. For each seed entity, SRTK executes a bounded beam search in the *space of relation paths*. Candidate paths are extended hop-by-hop by enumerating *next relations* available at the leaves reached by the current path. A special control token `END OF HOP` is appended at every step to allow early termination. The search is constrained by two hyperparameters: the *beam width* (maximum number of top-scoring path continuations retained per step) and the *maximum depth* (number of hops from the seeds).

Scoring and Aggregating Paths. At each expansion step, a dual-encoder scorer (Section 3.2.1) computes scores for all candidate next relations based on (i) the question text and (ii) the labels of the relations already traversed. Relation labels are obtained directly from Wikidata. Path scores are aggregated additively across hops, and only the top beams are retained according to the cumulative score. Upon reaching the depth limit, incomplete paths are finalized by appending `END OF HOP`, and the highest-scoring paths are selected.

Materialising Triplets and Applying Safety Limits. Each selected relation path is *materialized* into triplets by traversing from the seed entity along the path and recording edges (head, relation, tail) at every hop. To prevent excessive expansions, a limit on the number of discovered entities per seed is enforced; over-limit partial expansions are discarded. All triplets gathered across seeds and paths are de-duplicated.

3.2.3 Subgraph Construction

The retriever returns a flat set of RDF-style triples that (i) implicitly encode one–two hop neighborhoods and (ii) often include hub-driven, off-topic edges. Passing such unordered evidence to a lightweight LLM is both wasteful (token budget) and brittle (weaker grounding). We therefore introduce a *subgraph construction* stage that turns triples into compact, head-centric structures and a light *pre-reranking* step that suppresses fan-out and prioritizes question-aligned evidence. Each head’s subgraph is finally *compacted* into a short textual outline that preserves the key 1–2 hop justifications.

This design is guided by four principles: *preserve minimal justifications* (keep the shortest chains likely to support the answer), *control noise early* (prune high-fan-out branches), *promptability* (deterministic, token-efficient serialization), and *auditability* (persist a structured JSON alongside the compact text). Together, these steps bridge retrieval and generation by reconstructing multi-hop structure, suppressing irrelevant expansions, and formatting the remaining evidence for reliable use by small LLMs.

Given a question and its retrieved triples, we first enumerate simple one- and two-hop chains by building a subject-indexed adjacency and walking outward. Each emitted path is either a single edge (s, p_1, o_1) or a two-edge chain $(s, p_1, o_1) \rightarrow (o_1, p_2, o_2)$. Duplicate triples and paths are removed and trivial loops are discarded.

All identifiers are then mapped to human-readable labels (with safe fallback to IDs). Labeled paths are grouped by their *head* (the subject of the first edge) to materialize head-centric subgraphs: the unique nodes and edges present in that head’s paths, the de-duplicated path set itself, and convenience statistics (**#nodes**, **#edges**, **#paths**). These structured subgraphs are subsequently used by the pre-reranking/filtering pass and converted into compact, token-efficient outlines for the generator, while the full JSON remains available for diagnostics.

For efficiency, we implemented an additional labeling module: all unique QIDs from the triplets are collected and checked against a local mapping file (JSON key–value map from QID to label). If a QID is found, its label is taken directly from this map instead of querying a SPARQL endpoint. This optimization significantly speeds up the pipeline by avoiding redundant remote lookups. The mapping file is included in the supplementary artifacts of this work.

3.2.4 Pre-Reranking and Pruning

Given the head-centric subgraphs, we apply a lightweight yet effective pipeline to (i) prioritize subgraphs that are aligned with the question, (ii) trim hub-driven branches, and (iii) produce compact textual contexts suitable for a small LLM. The process has four stages.

(1) Pre-reranking of heads. For each head subgraph g and question q , we compute three signals: (i) a *semantic* score via bag-of-words cosine between the question and a deterministic subgraph stub $t(g)$ (vector-space IR [MRS08; SB88]); (ii) a *seed-overlap* score that measures coverage of question entities within the subgraph nodes (Jaccard-style overlap); and (iii) an *informativeness* score combining relation diversity and edge density (diversity-aware ranking and network density heuristics [New10]):

$$\text{sem}(g) = \cos(\text{BoW}(q), \text{BoW}(t(g))), \quad \text{seed}(g) = \frac{|V(g) \cap S(q)|}{\max(1, |V(g)|)},$$

$$\text{info}(g) = \frac{1}{2} \frac{|\{\text{predicates in } g\}|}{|E(g)|} + \frac{1}{2} \min\left(1, \frac{|E(g)|}{|V(g)|}\right).$$

Each component is min–max normalized over heads for the current question and linearly combined $\text{score}(g) = w_{\text{sem}}\text{sem}_n(g) + w_{\text{seed}}\text{seed}_n(g) + w_{\text{info}}\text{info}_n(g)$. We then retain heads that are within a fraction τ of the best score (“fraction-of-best”) and keep the top K by score with a tie-break in favor of smaller subgraphs (fewer edges). This step sharply reduces off-topic heads while preserving plausible evidence.

(2) Path-level pruning inside a head. Within each kept head, we rank its 1–2 hop paths by a path stub cosine penalized for fan-out:

$$\text{pScore}(\pi) = \cos(\text{BoW}(q), \text{BoW}(t(\pi))) - \lambda \overline{\log(1 + f(u, p))},$$

where $f(u, p)$ is the local fan-out for the pair (u, p) along the path and the bar denotes averaging over edges in π . We keep only the top M paths and drop edges not covered by them. Optionally, for a small list of known explosive predicates, we cap per-subject fan-out (at most c edges for a fixed (u, p)); in our experiments this hook is disabled by default, as the path-based pruning already contains hub explosion.

(3) Compaction to promptable text. Each trimmed head-centric subgraph is rendered into a short outline that preserves essential 1–2 hop justifications while removing duplicates. For every first-hop relation p_1 we aggregate its top objects $\{o_1\}$ (up to O_1) and select a single *pivot* o_1^* that has the richest second-hop neighborhood. From that pivot we expose up to P_2 distinct p_2 relations and, for each, up to O_2 objects $\{o_2\}$, ranked by frequency (lexical tie-breaks). The final textual form is deterministic and compact:

$$[\text{HEAD}] \ h; \quad h \text{ -- } p_1 \text{ -- } \{o_1, \dots\}; \quad \rightarrow \text{ [from pivot: } o_1^*] \quad p_2 \text{ -- } \{o_2, \dots\}.$$

This representation balances faithfulness and brevity under strict context budgets. For auditability, we persist the structured JSON (nodes, edges, paths, scores) alongside the compact text.

Hyperparameters and defaults. We expose a small set of knobs: K (top heads), τ (fraction-of-best), $(w_{\text{sem}}, w_{\text{seed}}, w_{\text{info}})$ (component weights), M (top paths per head), λ (fan-out penalty), and (O_1, P_2, O_2) (compaction budgets). Unless otherwise noted, we use moderate values that favor stability and compactness, and we disable predicate-specific capping by default.

3.2.5 Query-Focused Summarization and Answer Prediction

Having produced token-efficient, head-centric outlines in Sec. 3.2.4, we convert them into a single, question-targeted grounding and then into a final answer.

Query-focused summarization. For a question q and its compact outlines $\{c_h\}$ (ordered by the pre-reranking score and truncated to a global token budget), an LLM is prompted to generate a brief synopsis that is (i) *faithful* to $\{c_h\}$, (ii) *focused* on information that directly supports answering q , and (iii) *concise* (a short list of declarative facts). Surface forms of entities and relations are preserved to facilitate subsequent extraction.

Answer prediction. A second call conditions on q and the synopsis to return the final answer as verbatim spans (or a comma-separated list for multi-answer questions). If the synopsis is insufficient or contradictory, the model must abstain using a fixed, pre-declared token. This evidence-only constraint mitigates hallucination and yields calibrated “no answer” outcomes when the retrieved subgraphs do not contain the required facts.

4 Results

This chapter presents the experimental setup and results obtained using the methods described in Chapter 3. The evaluation is structured around the three research questions introduced in Chapter 1, covering: (i) comparisons between the proposed subgraph-based retrieval pipeline and baseline RAG approaches, (ii) the impact of retriever quality on final answer performance, and (iii) the effect of two-stage reranking compared to a simpler triple-based pipeline. For each experiment, the relevant setup, evaluation metrics, and quantitative as well as qualitative findings are reported. Results are interpreted with respect to answer correctness, context quality, and practical considerations observed during execution.

4.1 Experimental setup

4.1.1 Development Setup

The implementation of the project was carried out in multiple phases, each requiring specific tools and configurations. Despite the differences between the components, several development practices and technologies were consistently employed across all stages:

- **Git:** Used for version control and organizing the codebase for each project module.
- **Docker:** Employed to containerize the development environment, ensuring reproducibility and isolating dependencies.
- **PyCharm:** Served as the primary integrated development environment (IDE) to streamline Python development.
- **Local Computational Resources:** All experiments were run on a high-performance local workstation with the following specifications:

- GPU: NVIDIA GeForce RTX 5070 Ti
- CPU: AMD Ryzen 7 9800X3D @ 5.2GHz
- RAM: 32GB DDR5 (2×16GB, 6000 MHz)
- Storage: 2TB NVMe M.2 PCIe 4.0 SSD (7450 MB/s read, 6900 MB/s write)
- **Virtual Environments:** Python environments were isolated using conda, and dependencies were managed via `conda` and `pip`, enabling different project components to maintain separate library versions without conflicts.

4.1.2 System Setup

To support local retrieval and generation workflows, we install and run a set of foundational services. These include a local large language model (LLM) server via Ollama and a SPARQL-compatible graph database for structured retrieval.

LLM and Embedding Models

A deliberate design choice in this work is to perform all local inference and embedding generation using a *lightweight, open-weight LLM* deployed via **Ollama**¹. While larger proprietary models would likely achieve higher performance, this study investigates how far retrieval-generation quality can be pushed under constrained model capacity, with the additional benefits of fully local LLM inference, eliminating dependence on third-party APIs for generation and embedding

The model selected for the experiments is `llama3:8b-instruct` (referred to here as `llama3.1:8b`), based on the Meta AI LLaMA 3 architecture. Despite its relatively small parameter count (8B), it offers a favorable trade-off between computational cost and general-purpose reasoning ability, making it feasible to run on a single GPU. This constraint is intentionally embraced to:

- Explore whether retrieval and context optimization techniques (e.g., subgraph aggregation, reranking, pruning) can compensate for model size limitations.

¹<https://ollama.com/>

- Enable fully offline operation without reliance on cloud APIs or external compute clusters.
- Provide a replicable setup that can be reproduced on commodity hardware.

The model is installed using:

```
ollama run llama3:8b
```

Once downloaded, it is served locally at `http://localhost:11434`, with the same instance used for all generation and embedding tasks across the evaluated pipelines: (i) fuse contexts and answers generation in proposed SubgraphRAG, (ii) document ranking, vector index construction and answer generation in NaiveRAG, (iii) embedding-based retrieval and answer generation in LightRAG, and (iv) answer correctness evaluation with RAGAS.

For all inference-time dense retrieval and reranking steps, we use the `nomic-embed-text` model, served locally via Ollama. Developed by *Nomic AI*, it produces high-quality sentence-level embeddings optimized for semantic similarity tasks.

The model is launched using:

```
ollama run nomic-embed-text
```

Although all LLM inference in this work is performed locally, the retriever relies on sentence representations computed with a Hugging Face encoder. In all reported experiments we use `FacebookAI/roberta-base`². This choice is motivated by two considerations: (i) ease of integration—SRTK [She23] provides out-of-the-box support for Hugging Face encoders via simple parameter specification; and (ii) comparability with the reference system—the original SRTK retriever was trained with a RoBERTa backbone, so adopting RoBERTa-base ensures an apples-to-apples comparison with the baseline.

SPARQL Endpoint Setup

To support symbolic retrieval over the Wikidata knowledge graph, a local SPARQL endpoint is deployed using the `qEndpoint` container³. This service provides access to

²<https://huggingface.co/FacebookAI/roberta-base>

³<https://github.com/the-qa-company/qEndpoint>

a preloaded Wikidata "Truthy" dump and enables efficient subgraph extraction via SPARQL queries.

Following the official installation instructions, the endpoint is set up using Docker:

```
sudo docker run -p 1234:1234 --name qendpoint-wikidata qacompany
  /qendpoint-wikidata
sudo docker start qendpoint-wikidata
```

The downloaded image contains an embedded Wikidata dump of approximately 50GB, allowing for entirely local execution without reliance on external SPARQL services.

To simplify query syntax, standard Wikidata prefixes are added to the container:

```
wget https://raw.githubusercontent.com/the-qa-company/qEndpoint/
  master/wikibase/prefixes.sparql
sudo docker cp prefixes.sparql qendpoint-wikidata:/app/qendpoint
  && rm prefixes.sparql
```

The resulting endpoint is accessible at `http://localhost:1234/sparql` and is used throughout the symbolic retrieval pipeline to extract graph neighborhoods around question entities.

The use of `qEndpoint` in this thesis complies with its GNU General Public License v3.0 (GPLv3), which permits non-commercial academic use and modification of the software, as long as license conditions are met and proper attribution is provided.

4.1.3 Dataset

As outlined in Section 3.1, we use the WebQSP-WD dataset [SG18] as the primary benchmark for our experiments. Prior to evaluation, the dataset was cleaned and pre-processed. This included renaming fields, removing irrelevant attributes, and enriching missing information through external resources such as entity recognition and linking, as well as Wikipedia–Wikidata mapping. We also filtered out examples with empty answers or answers not corresponding to Wikidata entities (e.g., date values).

Table 4.1 summarizes the number of valid examples retained in the final training and test sets after preprocessing.

Table 4.1: Number of valid examples before and after preparing

	Train Split	Test Split
Original dataset size	3,098	1,033
Valid after preparation	2,764	1,023
Percentage retained (%)	89.2%	99.0%

In addition to preparing the cleaned dataset, we construct a supplementary corpus of unstructured text to support document-based retrieval experiments. This corpus consists of Wikipedia articles associated with the entities mentioned in the test questions. Specifically, we extract articles corresponding to all unique Wikidata Q-IDs found in the `question_entities` field of the test split (681 in total). These documents serve as the knowledge base for indexing in downstream RAG pipelines vector and graph-based approaches.

To build this corpus, we first resolve each Q-ID to one or more candidate Wikipedia titles using the `wikimapper` tool. The resulting titles are then expanded using simple normalization rules (e.g., lowercase variants, replacing underscores with spaces). In the first stage, we search for these titles in a static English Wikipedia snapshot provided by Hugging Face (`wikimedia/wikipedia`, November 2023)⁴. This yields 615 articles.

For the remaining Q-IDs not matched in the offline dump, we fall back to the official Wikipedia API. This dynamic stage retrieves an additional 177 articles. In total, we obtain 792 documents, with some Q-IDs mapped to multiple relevant articles (e.g., due to redirects or disambiguation). This final corpus serves as the unified text base for all document-level retrieval experiments.

Table 4.2 summarizes the article retrieval process.

Table 4.2: Wikipedia article retrieval statistics

Retrieval Stage	Number of Articles
Articles from Hugging Face dump	615
Articles from Wikipedia API	177
Total articles retrieved	792

⁴<https://huggingface.co/datasets/wikimedia/wikipedia/viewer/20231101.en>

4.2 Document-Based Baselines: Naive and Advanced RAG Pipelines

To establish a reliable baseline for comparison with our subgraph retrieval approach, we implement and evaluate standard document-centric RAG pipelines. These baselines operate over a corpus of Wikipedia articles, which serve as the primary source of textual knowledge.

As described in Section 3.1.3, we constructed a compact, task-specific document collection consisting of 792 Wikipedia articles. This corpus was curated by extracting the unique entities extracted from the `question_entities` field in the test portion of the dataset. The resulting data set provides a grounded semantically relevant context for the questions and serves as the foundation for the construction of vector and graph indexes.

Using this corpus, we build both a naive RAG pipeline and an advanced graph-based baseline using LightRAG [Guo+25]. The naive pipeline follows the standard RAG paradigm: retrieving relevant documents based on the input question using a dense vector index, then generating an answer using a language model conditioned on the retrieved content. In contrast, the LightRAG baseline combines document retrieval with a graph-based expansion and reasoning component, allowing it to incorporate structural knowledge during generation. The next sections describe in detail the implementation and evaluation of both variants.

4.2.1 Naive RAG Implementation

To establish a document-level retrieval baseline, we implement a naive Retrieval Augmented Generation pipeline [Gao+24]. In this setup, question answering is performed based on semantically relevant Wikipedia articles, without incorporating any graph-structured knowledge or symbolic supervision.

The pipeline begins with the construction of a dense vector index over the document corpus. Documents are segmented into overlapping chunks using the default node parsing strategy of LlamaIndex, which tokenizes text into semantically coherent units. In our implementation, we rely on the default `SentenceSplitter` configuration: a chunk size of 1024 tokens with an overlap of 200 tokens. Each chunk is embedded using the local

`nomic-embed-text` model, and the resulting vectors are stored in a persistent vector database optimized for approximate nearest neighbor (ANN) search. This index enables fast and scalable top- k retrieval.

At query time, a semantic retriever fetches the top- k most relevant text chunks for a given question based on cosine similarity in embedding space. The retrieved chunks are concatenated to form a context block, which is then passed together with the question into an instruction-tuned LLM for answer generation. We employ our custom answering prompt (see Appendix C), which is identical across all RAG implementations to ensure comparability. The model is instructed to generate an answer strictly based on the provided context; if the context lacks sufficient information, the expected output is `NO_ANSWER`.

4.2.2 Advanced GraphRAG Baseline: LightRAG Index Construction

To implement an advanced RAG baseline, we adopt the LightRAG framework [Guo+25], which extends traditional RAG systems by integrating symbolic graph structures into both indexing and retrieval. This hybrid architecture is designed to overcome limitations of flat representations by enhancing contextual awareness and capturing inter-entity relationships during retrieval.

The index construction proceeds in two stages. First, a `LightRAG` instance is initialized in a dedicated working directory. This step configures the system’s internal components, including storage backends for vector and graph indices, embeddings and the LLM model, and default parameters for text segmentation and graph generation.

Second, a collection of preprocessed documents iteratively inserted into the index via the asynchronous method `ainsert`. This method performs three internal operations per document:

- **Text chunking and embedding:** The document is segmented into overlapping spans, and each span is embedded into a dense vector representation using the configured embedding model.
- **Graph construction:** Entities and semantic relations are extracted from the text and used to construct a lightweight graph structure that encodes interconnections between key concepts.

- **Dual-level indexing:** Both vector embeddings and the extracted graph are stored persistently, enabling subsequent dual-mode retrieval that combines semantic similarity with topological relevance.

To improve reproducibility and efficiency, previously processed documents are recorded in a log file. On subsequent runs, any matching files are automatically skipped, avoiding redundant insertions and reducing runtime. The overall process is parallelized using `asyncio` coroutines and tracked with progress indicators for monitoring.

Computational Cost. The index construction process is computationally intensive and resource-demanding. In this study, the full indexing pipeline was executed on a high-performance desktop machine equipped with a single NVIDIA RTX 5070 Ti GPU. Processing the entire collection of Wikipedia articles required over 60 hours of wall-clock time. The resulting graph-based index comprised 107,812 nodes and 56,692 edges, reflecting the substantial scale and granularity of the extracted knowledge representation.

4.2.3 Answers Generation with LightRAG

Following the construction of the graph-augmented index, we employ the LightRAG framework to generate answers to natural language questions. Each question is submitted asynchronously via the `aquery()` method, and results are logged for further evaluation. Previously answered questions, identified via a checkpointed output log, are automatically skipped to optimize computation time. In addition to answer generation, we also retrieved the underlying contexts for evaluation purposes, which required additional configuration of the query parameters and a separate call to `aquery()` in a context-only mode.

Retrieval Modes. LightRAG supports several query modes that define how relevant knowledge is retrieved from the indexed graph-text structure:

- **naive:** Retrieves top- k nodes based solely on embedding similarity; does not use the graph structure.
- **local:** Initiates symbolic traversal from semantically close nodes, limited to a local neighborhood in the graph.

- **global**: Expands the search to a broader portion of the graph, enabling access to semantically distant but potentially relevant nodes.
- **hybrid**: Combines local traversal with semantic reranking to improve context relevance.
- **mix**: Aggregates results from multiple retrieval strategies (naive, local, global), followed by a reranking step.
- **bypass**: Disables retrieval and returns a placeholder; useful for debugging or ablation studies.

In this work, we used the **hybrid** mode as our primary retrieval configuration, as it combines the focused neighborhood exploration of **local** retrieval with the broader semantic matching of **global** retrieval. This approach offered a good balance between coverage and contextual relevance, while avoiding the complexity of multi-strategy aggregation in **mix** mode.

4.3 Retriever Modeling and Evaluation

This section investigates how retriever quality and the resulting extracted subgraphs affects end-to-end RAG performance. We proceed from data construction under weak supervision to model training and validation diagnostics, then compare multiple retriever variants and assess their end-to-end impact under identical generation settings.

4.3.1 Preprocessing: Weak Supervision for Relation Selection

We construct step-level supervision from question–answer pairs by deriving multi-hop paths in the knowledge graph and forming positives (follow this relation) and distractors (negatives) at each traversal step. We ablate the path scoring function—*jaccard* (token overlap between the question and relation labels) vs. *recall* (coverage with respect to answer entities)—while fixing the number of sampled negatives and the selection threshold.

Metric ablation (fixed negatives and threshold). Under identical settings ($n_{\text{neg}} = 15$, $\tau = 0.5$), the *jaccard*-based preprocessing yields **8,556** training instances, whereas the *recall*-based variant yields **10,206**. The difference is expected: *recall* is less lexically selective and accepts a broader set of candidate paths that reach (or partially cover) the answer entity set, resulting in a larger but potentially noisier training signal; *jaccard* is stricter with respect to lexical alignment and therefore admits fewer paths at the same threshold.

Extended Jaccard configuration (increased supervision). Given the substantially lower graph noise observed with *jaccard* at $\tau=0.5$ compared to *recall* (see retrieval analysis in Sec. 4.3.3), we additionally construct a higher-signal variant that preserves lexical anchoring while enlarging the training signal: **jaccard with 30 negatives per step and a relaxed threshold $\tau=0.4$** . This configuration is intended to test whether increasing supervision under the *jaccard* scoring function can recover some of the coverage benefits seen with *recall*, without fully incurring its graph-size inflation.

Table 4.3: Preprocessing outcomes.

Scoring metric	Negatives / step	Threshold	# Training instances
<i>jaccard</i>	15	0.5	8,556
<i>recall</i>	15	0.5	10,206
<i>jaccard</i>	30	0.4	10,445

Configuration summary and forward reference. In total, we carry forward three training-data configurations for downstream comparison:

- **Jaccard (15, $\tau=0.5$)** — smaller, lexically selective dataset.
- **Recall (15, $\tau=0.5$)** — larger dataset due to broader acceptance of answer-reaching paths.
- **Jaccard (30, $\tau=0.4$)** — extended supervision under *jaccard* (more negatives, lower threshold).

We select *jaccard*-based preprocessing as the *primary* strategy due to its superior size–efficiency at retrieval time under realistic context budgets (Sec. 4.3.3). The extended *jaccard* variant is included as a high-coverage alternative. The retrieval-level effects of all three

datasets (coverage vs. subgraph size) and their end-to-end impact on answer correctness are presented in Secs. 4.3.3 and 4.4, respectively.

Data exclusions and endpoint issues. Across all runs, a subset of questions in the training split had no valid graph paths connecting the question entities to the annotated answer entities and were therefore skipped during path extraction. In all runs, **628** out of **2,764** questions ($\approx 22.7\%$) lacked any connecting paths and were excluded from training. In addition, we observed a reproducible SPARQL error (`QueryBadFormed`) when probing relations around the entity `Q1974` (British Columbia). The endpoint responded with `HTTP 400` and indicated a malformed query; we conservatively excluded the affected cases from preprocessing. While these exclusions reduce the effective training set size, they reflect realistic limitations of KG coverage and endpoint robustness and are accounted for in our dataset statistics.

4.3.2 Training Setup and Hyperparameter Study

We fix the encoder to `FacebookAI/roberta-base` and the loss to `cross_entropy`. This keeps the setup simple and directly comparable to the SRTK reference, and it matches our supervision format (“one positive + N negatives” per step). Contrastive objectives add extra moving parts (special margins/temperatures, negative mining), which would blur the effects we want to study. Accordingly, we vary only the learning rate, batch size, and number of epochs exposed by the `srtk` CLI:

- `--model-name-or-path (-m)`: Hugging Face encoder identifier or local path.
- `--learning-rate (-lr)`: optimizer step size.
- `--batch-size`: number of instances per optimization step.
- `--loss`: `cross_entropy` (multi-class over candidates per step) or `contrastive`.
- `--max-epochs`: upper bound on training epochs.

Diagnostics vs. selection. During training we monitor training/validation curves (loss) to detect underfitting/overfitting and optimization instabilities. However, the *final model selection* is made by **retrieval quality** on a held-out evaluation split under a fixed retrieval configuration (**beam width** = 6, **max depth** = 2). For each model we run the retriever on the test split and compute hit rate (Recall):

$$\text{Recall} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbf{1}[a^*(q) \in \mathcal{E}(q)].$$

Here, $a^*(q)$ denotes the gold answer entity (QID) for question q , and $\mathcal{E}(q)$ is the set of entities appearing in any retrieved subgraph for q under the fixed retrieval configuration. We also log the average subgraph size (number of triplets) to track noise. The selected checkpoint maximizes Recall; in case of ties we prefer the model with a smaller average subgraph size.

Initial configuration and datasets. We start from a standard setup:

Model: FacebookAI/roberta-base **Batch size:** 16 **Learning rate:** 2×10^{-5} **Loss:** CrossEntropyLoss **Epochs:** 10

and train on two preprocessing variants with fixed negatives/threshold:

- **Jaccard** ($n_{\text{neg}}=15, \tau=0.5$)
- **Recall** ($n_{\text{neg}}=15, \tau=0.5$)

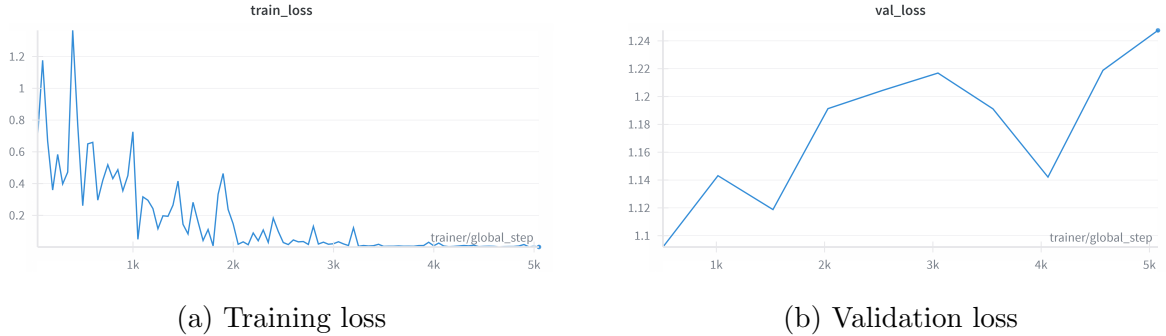


Figure 4.1: Loss behavior for **Jaccard** ($n_{\text{neg}}=15, \tau=0.5$)

Under identical hyperparameters (**batch size** = 16, **learning rate** = 2×10^{-5} , 10 epochs), the **Jaccard** configuration (Figs. 4.1) exhibits a rapid decrease of training loss

4 Results

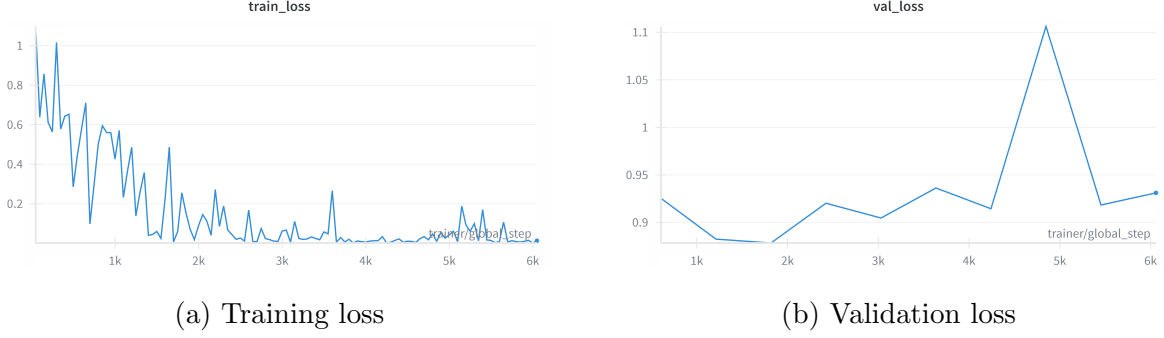


Figure 4.2: Loss behavior for **Recall** ($n_{\text{neg}}=15$, $\tau=0.5$)

followed by a steady increase of validation loss from ~ 1.10 to ~ 1.24 . This pattern indicates overfitting: the model fits the training data while generalization on the held-out split deteriorates. The **Recall** configuration (Figs. 4.2) shows a milder trajectory: training loss declines and validation loss follows a shallow U-shape with an early minimum around 0.88–0.90 and moderate fluctuations thereafter. Although this behavior is comparatively healthier, by epoch 10 both settings still display signs of overfitting.

These diagnostics motivate reducing the effective training budget and adjusting optimization hyperparameters. In the sequel, we therefore (i) vary the learning rate and the number of epochs, and (ii) select checkpoints *post hoc* by retrieval performance on a held-out split (hit/miss under fixed beam width and depth), rather than by loss alone. This aligns model selection with the downstream objective while mitigating overfitting observed in the initial runs.

Low-budget run (Jaccard, 1 epoch, $\text{LR} = 10^{-5}$). Even with a reduced budget, the model overfits early: the training loss drops, while the validation loss increases almost monotonically. Thus, simply shortening the training with a lower learning rate does not improve the generalization of *jaccard* (15 , $\tau=0.5$).

Recall ($\text{LR} = 2 \times 10^{-5}$, 3 epochs, $n_{\text{neg}}=15$, $\tau=0.5$). Training loss descends with moderate oscillations; validation loss shows a shallow U-shape with a minimum near ~ 0.86 then a slight uptick. This is healthier than the 10-epoch runs; the best checkpoint is expected mid-training, which we select *post hoc* by retrieval hit rate (beam=6, depth=2).

4 Results

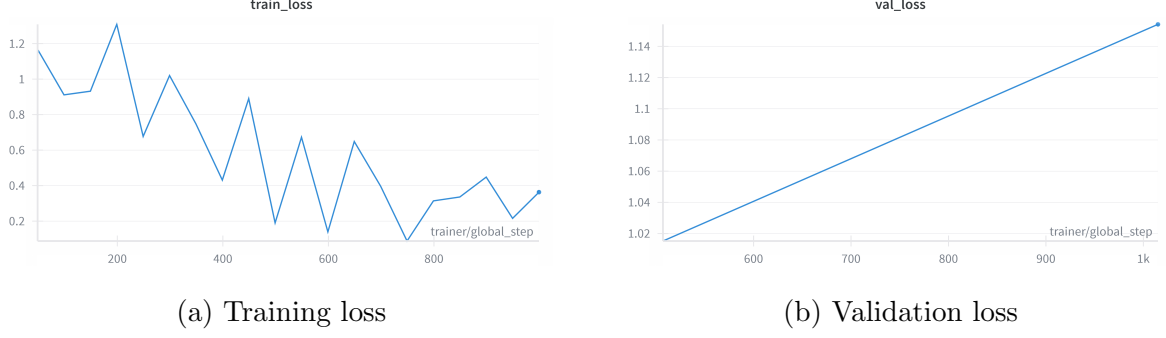


Figure 4.3: Loss behavior for Jaccard, 1 epoch, $\text{LR} = 10^{-5}$

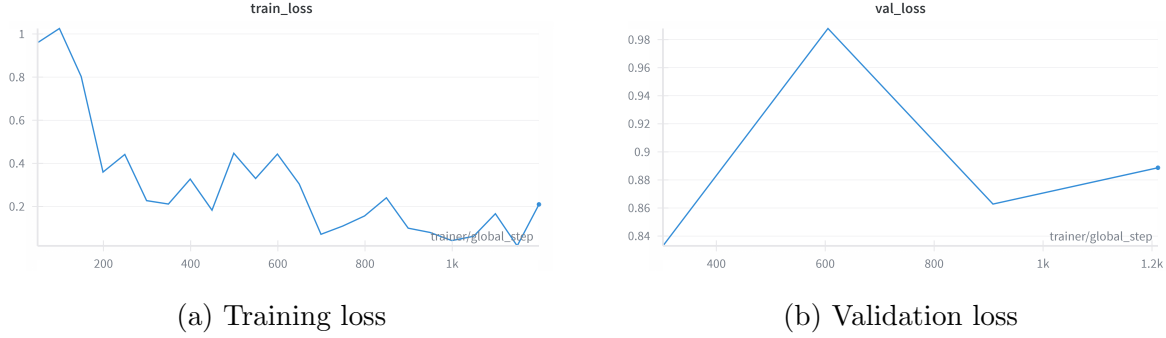


Figure 4.4: Loss behavior for Recall, 3 epoch, $\text{LR} = 2 \times 10^{-5}$

Extended Jaccard ($\text{LR} = 2 \times 10^{-5}$, 3 epochs, $n_{\text{neg}}=30$, $\tau=0.4$). With stronger supervision, training loss stabilizes at low values; validation exhibits an early dip (~ 0.96), a mid-run rise (~ 1.08), and a late recovery (~ 1.00). Although validation remains above the recall setup, we carry this model forward because its dataset achieves the highest retrieval coverage (Sec. 4.3.1); checkpointing follows the same retrieval-based criterion.

Overall, this run converges but does not reach the lower validation levels observed with the *recall* configuration; nevertheless, we carry it forward because the underlying dataset delivers higher retrieval coverage (Sec. 4.3.1).

4.3.3 Retriever Comparison and Selection

Across five training runs, the retrieval Recall remains in a narrow band (0.729–0.754), while the average subgraph size varies 4–5 $\times$. Given 1,023 evaluation questions, differences below 2.6 percentage points are not statistically significant. Thus, under fixed

4 Results

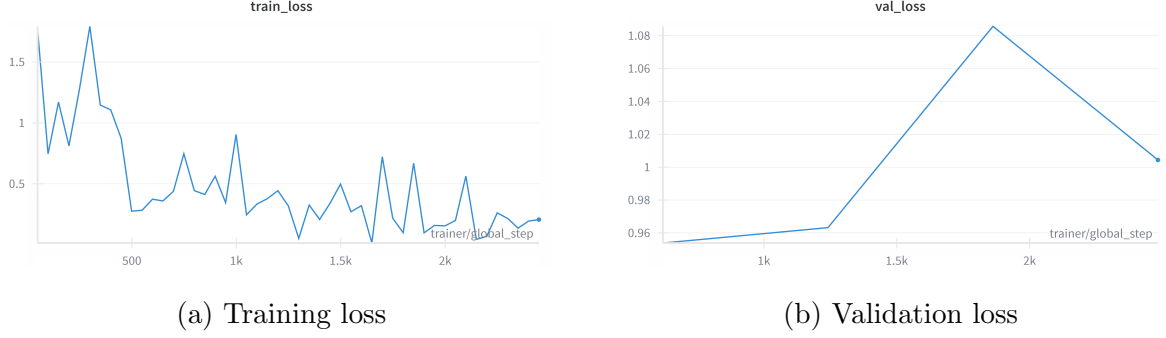


Figure 4.5: Loss behavior for Jaccard 30 negatives 0.4 threshold, 3 epoch, $LR = 2 \times 10^{-5}$

beam/depth the dominant factor is the coverage-size trade-off rather than the specific training schedule.

Table 4.4: Training profiles and retrieval performance on the test split (evaluated with beam=6, depth=2).

Dataset config	LR	BS	Epochs	Retrieval Recall	Avg triplets
Jaccard (15, 0.5) (M1)	2×10^{-5}	16	9	0.737	31.48
Recall (15, 0.5)	2×10^{-5}	16	9	0.753	129.53
Jaccard (15, 0.5)	1×10^{-5}	16	1	0.729	30.93
Recall (15, 0.5)	2×10^{-5}	16	3	0.743	132.88
Jaccard (30, 0.4) (M2)	2×10^{-5}	16/32	3	0.754	143.67

Pretrained SRTK scorer. As a baseline, we also evaluate the (drt/srtk-scorer)⁵, a RoBERTa-based model released with SRTK and trained on WebQSP (Freebase), SimpleQuestionsWikidata, and SimpleDBpediaQA. Under the *same* retrieval configuration as our models (beam width = 6, max depth = 2), it attains:

$$\text{Recall} = 0.658 \text{ (673/1023)}, \quad \text{avg. triplets} = 32.18,$$

Compared to our *balanced* Jaccard retriever (15 negatives, $\tau=0.5$; Recall = 0.737 at 31.48 triplets), the pretrained scorer is lower by **7.9 p.p.** in coverage with a similar subgraph size. The gap remains against our *high-coverage* Jaccard variant (30 negatives, $\tau=0.4$; Recall = 0.754) and the *recall*-based model (15, $\tau=0.5$; Recall = 0.753). Given $N=1023$ questions, the difference between 0.658 and 0.754 is statistically significant.

⁵<https://huggingface.co/drt/srtk-scorer>

Interpretation. This result suggests that task- and graph-specific weak supervision (our preprocessing on Wikidata with multi-hop paths) yields better ranking for depth-2 retrieval than a generic pretrained scorer. The SRTK model card notes a bias toward 1-hop behavior on SQ/Wikidata and DBpedia, which can further limit performance at depth 2. We therefore carry forward our trained retrievers for downstream RAG experiments and treat `drt/srtk-scorer` as an out-of-the-box baseline.

4.3.4 Beam Width and Depth Sensitivity Analysis

Table 4.5: Beam/depth ablation for two retrievers. M1: Jaccard (15, 0.5), LR= 2×10^{-5} , 9 epochs; M2: Jaccard (30, 0.4), LR= 2×10^{-5} , 3 epochs. Retrieval on the same split.

Retriever	Beam width	Max depth	Recall	Avg. triplets	Efficiency (Recall / 100 triplets)
M1 (balanced)	2	1	0.625	9.10	6.86
M1 (balanced)	2	2	0.657	10.69	6.15
M1 (balanced)	6	2	0.737	31.48	2.34
M1 (balanced)	10	2	0.763	56.77	1.34
M1 (balanced)	10	4	0.750	57.32	1.31
M2 (high-coverage)	6	2	0.754	143.67	0.53
M2 (high-coverage)	10	2	0.770	195.76	0.39

Table 4.5 reports a sensitivity analysis of *beam width* and *maximum depth* for two retrievers: a **balanced** model (M1; *jaccard*, $n_{\text{neg}}=15$, $\tau=0.5$, LR = 2×10^{-5} , 9 epochs) and a **high-coverage** model (M2; *jaccard*, $n_{\text{neg}}=30$, $\tau=0.4$, LR = 2×10^{-5} , 3 epochs). On the shared evaluation split, *beam width* is the primary driver of coverage. Increasing the beam from 6 to 10 at depth 2 improves recall from 0.737 to 0.763 for M1 and from 0.754 to 0.770 for M2. This gain, however, is accompanied by an almost linear increase in subgraph size: M1 grows from approximately 31.5 to 56.8 triplets on average, and M2 from approximately 143.7 to 195.8 triplets.

Conversely, narrowing the search to `beam=2` sharply degrades coverage on M1 (recall 0.625–0.657), confirming that a narrow beam prunes answer-bearing paths. Varying *depth* shows that most answers are recovered within 1–2 hops: for M1, moving from depth 1 to 2 at `beam=2` yields only a modest +3.2 p.p. (0.6246→0.6570), indicating that depth-2 adds a thin but useful layer of hits on top of predominantly 1-hop answers. Pushing deeper is counterproductive: at `beam=10`, increasing depth from 2 to 4 reduces recall (0.763→0.750) with virtually unchanged graph size, suggesting that additional branches introduce distractors that the scorer cannot reliably down-weight.

To quantify the cost–benefit trade-off, we report an *efficiency* measure (recall per 100 triplets). Compact settings are markedly more efficient—for M1 at `beam= 2/depth 1` the efficiency is roughly 6.9—whereas wide-beam regimes collapse to about 1.3 (M1, `beam= 10/depth 2`) and as low as 0.4 (M2, `beam= 10/depth 2`). Given the strict context budget of our small LLM, these results delineate a clear *coverage–size Pareto frontier*: wider beams surface more answers but at a rapidly increasing cost in retrieved evidence.

For downstream RAG evaluation, we therefore carry forward two representative points on this frontier: R-BALANCED (M1, `beam= 6, depth 2`; recall 0.737 with ~ 31.5 triplets) as a realistic, size-efficient configuration, and R-HIGH (M2, `beam= 10, depth 2`; recall 0.770 with ~ 195.8 triplets) as a high-coverage, high-noise configuration. The former reflects practical deployment constraints; the latter is expressly used to test whether our two-stage reranking and context selection can convert additional coverage into higher end-to-end answer correctness under a fixed token budget.

4.4 End-to-End Evaluation with RAG

To evaluate the full pipeline, we follow the procedure described in Sec. 3.2.3, Sec. 3.2.4, and Sec. 3.2.5. For both R-BALANCED and R-HIGH, subgraphs were constructed and pruned with identical hyperparameters, while the query-focused summarization varied in budget and timeout. This setup allows us to assess the trade-off between conciseness and recall in the final answers.

Subgraph construction and pruning. We used the following default parameters for both:

- **Top- K heads:** 8 candidate head subgraphs retained.
- **Fraction-of-best cutoff:** $\tau = 0.70$, heads below 70% of the best score discarded.
- **Score weights:** semantic $w_{\text{sem}} = 0.55$, seed-overlap $w_{\text{seed}} = 0.20$, informativeness $w_{\text{info}} = 0.25$.
- **Per-subject cap:** at most 2 edges for the same (u, p) pair.
- **Path pruning:** top 4 ranked paths per head.

- **Fan-out penalty:** $\lambda = 0.3$.

Query-focused summarization. For the final fusion of compact outlines into question-specific contexts, we adopted two distinct settings:

- **R-balanced (compact):** up to 6 head snippets, 120-word synopsis target, 60s timeout.
- **R-high (expanded):** up to 10 head snippets, 140-word synopsis target, 90s timeout.

Summary of configurations. Table 4.6 summarizes the parameter choices for both variants.

Parameter	R-balanced	R-high
Top- K heads	8	
Fraction cutoff τ	0.70	
Weights ($w_{\text{sem}}, w_{\text{seed}}, w_{\text{info}}$)	(0.55, 0.20, 0.25)	
Cap per subject	2	
Top paths per head	4	
Fan-out penalty λ	0.3	
Max heads (summarization)	6	10
Max words (synopsis)	120	140
Timeout (s)	60	90

Table 4.6: Configuration of subgraph pruning and query-focused summarization for R-balanced and R-high. Shared parameters are listed jointly; summarization budgets differ across variants.

4.5 Comparative Evaluation with Baselines

To fairly assess the effectiveness of our proposed pipeline, we conduct a comparative evaluation against the baseline systems described in Sec. 4.2. For comparability, all systems are prompted with the same *answer prompt*, which is documented in Appendix C.

NaiveRAG. Documents are segmented into chunks of 1024 tokens with an overlap of 200 tokens. The retriever returns the $\text{top-}k = 4$ chunks, resulting in an effective context window of approximately 4096 tokens.

LightRAG (hybrid mode). The parameter `top_k` in LightRAG controls the number of candidate items retrieved *before* token-based truncation. Its semantics depend on the retrieval mode: in *local* mode, `top_k` corresponds to the number of entity descriptions (nodes) to retrieve, whereas in *global* mode, it corresponds to the number of relationship descriptions (edges). In the *hybrid* mode, both branches are combined, and the effective context is further constrained by `max_token_for_local_context` and `max_token_for_global_context`, which cap the total token budget for the local and global contexts, respectively. In our setup we set `top_k=60`, while explicitly limiting both branches to 2048 tokens each, resulting in a balanced effective context size of approximately 4096 tokens — directly comparable to the 4×1024 token chunks used in the NaiveRAG configuration.

We then compute the `answer correctness` score using RAGAS [Es+25] across all systems. In addition, we report the proportion of generated answers labeled as `NO_ANSWER`, i.e., cases where the retrieved context was insufficient and the model correctly abstained from answering, as instructed. The aggregated results are shown in Table 4.7, which includes both `answer correctness` and the `NO_ANSWER rate`.

Pipeline	Answer Correctness	NO_ANSWER Rate
R-BALANCED (OURS)	0.6252	0.0626
R-HIGH (OURS)	0.6324	0.0557
Naive RAG [Gao+24]	0.5358	0.0811
LightRAG [Guo+25]	0.5695	0.0674

Table 4.7: Final evaluation of our pipeline compared to baselines. We report both the RAGAS-based `answer correctness` and the proportion of abstentions labeled as `NO_ANSWER`.

Results. Our graph-based pipeline outperforms both baselines, achieving the highest answer correctness (0.6324) and the lowest `NO_ANSWER` rate (0.0557). Among our variants, the higher-recall configuration (R-HIGH) slightly surpasses R-BALANCED; a detailed analysis is deferred to the Discussion.

Evaluation note. During scoring, a small subset of rows failed in RAGAS due to output-parsing issues (LLM response interpretation), yielding `null` values for `answer`

correctness. We provide a patch script that re-evaluates only the missing rows and merges the results; aggregate numbers reported here are computed on the patched dataset. Both versions are included in the artifacts, with the final file suffixed as `.patched`.

5 Discussion

This chapter discusses the results of the proposed graph-based RAG pipeline, synthesizes the findings to answer the research questions (Sec. 1.2), and concludes with limitations and avenues for future research.

RQ1. Our experiments show that the choice and quality of the pretrained retriever directly affect end-to-end performance. The higher-coverage variant (R-HIGH) produced noisier graphs but—once processed by pre-reranking, pruning, and query-focused summarization—delivered the strongest downstream results, surpassing the more conservative R-BALANCED in answer correctness (63.24% vs. 62.52%) and NO_ANSWER rate (5.57% vs. 6.26%). Thus, under our pipeline, retrieval-time recall (i.e., ensuring gold-bearing evidence is present somewhere in the subgraph) outweighs the marginal cost of extra noise, which is largely mitigated by subsequent structuring and compression.

RQ2. Building on RQ1, the structuring layer is what converts high recall into usable signal under a fixed context budget. In practice, lightweight pre-reranking, pruning, and query-focused summarization preserve gold-supporting paths while preventing context precise, enabling the high-recall retriever to translate into higher answer correctness. For small, low-noise outputs (on the order of ~ 10 triples), a direct triple-to-context route might suffice; however, noisy graphs that might have higher recall (200+ entities/edges) would exceed token limits and yield off-topic generations if passed unordered. Although we did not run a dedicated experiment, qualitative observations across runs and standard design principles for context-limited generation indicate that passing large sets of unordered triples is likely either to exceed the context window or, more plausibly, to yield noisy, off-topic generations; we therefore regard compact head-centric structuring and selective path retention as necessary to obtain a reliable signal.

RQ3. Under identical experimental conditions, and a shared answer prompt, our pipeline attains the highest answer correctness (63.24%) and the lowest `NO_ANSWER` rate (5.57%), outperforming LightRAG (56.95%, 6.74%) and naive RAG (53.58%, 8.11%) in Table 4.7. These results indicate that graph-derived, query-focused contexts from a public KB can match or surpass document-centric RAG pipelines on end-to-end answer correctness using LLM-as-a-Judge approach (RAGAS), establishing a viable knowledge-graph-first alternative for open-domain QA.

5.1 Limitations And Future Work

Benchmark. One of the limitations concern temporal and alignment inconsistencies across the resources used for evaluation. The original WebQuestions/WebQSP/WebQSP-WD [Ber+13; Yih+16; SG18] annotations and gold answers reflect older snapshots, whereas Wikidata facts, Wikipedia articles, entity-page mappings evolve continuously; additionally, static linkers (e.g., QID→title maps) and mixed acquisition channels (frozen dumps vs. live APIs) may refer to different vintages. As a result, entity labels, page titles, and even factual statements can drift: a gold answer that was correct at annotation time may diverge from the state of the knowledge graph or articles at retrieval time, and entity-article alignments may become partial or stale. These mismatches directly affect measured correctness (and abstentions), potentially penalizing systems that return up-to-date answers or favoring those that happen to match historical golds.

Mitigations for future work include time-synchronizing all components (KG, article corpus, link maps) to a single snapshot; auditing or remapping entity-article alignments; and, where feasible, revalidating gold answers against the chosen temporal cut or adopting time-aware scoring. Without such controls, comparisons between structured and unstructured evidence remain confounded by cross-snapshot drift rather than purely methodological differences.

Evaluation metric. The LLM-as-judge score used in RAGAS (*answer correctness*) is sensitive to surface form: explicit abstentions may receive small non-zero scores, while answers that fully entail the gold but include related additional detail can be down-weighted. For example, the prediction “*Mobile, Alabama, United States*” was scored only ~ 0.56 against the gold “*Mobile*” for the query “*where is jamarcus russell from?*”,

despite being semantically correct; conversely, explicit `NO_ANSWER` outputs sometimes received ~ 0.1 instead of 0. These effects reflect formatting and grader heuristics rather than factual adequacy, and may bias cross-system comparisons.

Future evaluations could be made more robust by introducing a few refinements. Explicit abstentions should always score as 0, and answers should be lightly normalised (e.g., for place names) before grading. To reduce grader variance, results can be checked with multiple judge prompts or models, complemented by occasional human spot checks. In addition to correctness, it is useful to report the abstention rate and retain raw outputs for transparency. Finally, beyond answer scores, incorporating a small set of retrieval-focused metrics—such as context recall, context relevance, and faithfulness—would provide a more balanced view of system performance.

Scorer model training. In this work, we trained several retrieval models with different configurations and pre-training datasets. While their performance was reasonable (in the range of 65–77%), the results remained far from optimal. In contrast, the retrieval model employed in the referenced framework¹ achieved an answer coverage rate of 0.9728 on SimpleQuestionsWikidata [Die+17] (depth 1), which is substantially higher and close to ideal. Such a model, provided it is not excessively noisy, would make many of the optimization stages explored in this thesis largely redundant, since the retrieved triples could be directly supplied as context.

Future research should therefore investigate the integration of stronger retrieval models. With sufficiently robust retrieval performance, the pipeline could be simplified by reducing or even eliminating auxiliary optimization steps, shifting the focus toward efficient training data construction and systematic evaluation of retrieval noise.

Structured data optimization. The pruning and compaction pipeline developed in this work demonstrates that lightweight heuristics can reduce noise in head-centric sub-graphs and yield contexts that are both concise and usable by a small LLM. Nevertheless, the approach remains a relatively modest step within the broader landscape of structured data optimization. The methods rely on manually designed signals (semantic overlap, seed coverage, relation diversity) and simple path-level pruning, which, while effective in practice, cannot fully capture the semantic richness or task-specific nuances of the

¹<https://huggingface.co/drt/srtk-scorer>

underlying knowledge graph. Moreover, the current heuristics are sensitive to parameter choices (e.g., fan-out penalties, compaction budgets), limiting their generalizability across datasets and query types.

This work shows only a first step toward optimizing structured data for RAG. Many directions remain unexplored: more refined ways of filtering and ranking graph structures, alternative strategies for compacting them into text, or even entirely different approaches to representing structured knowledge. Another promising line of research is to apply the methodology beyond Wikidata. Although the current pipeline was tested solely on this source, its design is domain-agnostic and could be adapted to other large-scale open-domain graphs such as Freebase or DBpedia, as well as to specialised graphs in areas like biomedicine, finance, or law. Exploring such settings would help assess the generalisability of the approach and reveal domain-specific adjustments.

Finally, future systems may also move beyond static retrieval. Interactive or iterative mechanisms—potentially involving a human in the loop—could be used to clarify ambiguous entities or verify intermediate results before finalizing the context. This kind of interaction would be especially valuable in complex or high-stakes domains, where retrieval accuracy is critical. In this sense, the space for improvement is wide, and the current study should be seen as an initial contribution rather than a comprehensive solution.

6 Conclusion

The work presented in this thesis set out to explore and implement a Retrieval-Augmented Generation (RAG) pipeline driven by a structured knowledge graph as its primary source of information. Motivated by the limitations of large language models (LLMs) in accessing up-to-date, domain-specific, and factually consistent knowledge, the proposed system sought to leverage the structured nature of knowledge graphs to enhance retrieval precision and answer reliability.

The central contribution lies in the design, integration, and evaluation of an end-to-end architecture capable of transforming a natural language question into a final answer through a series of interlinked components: entity linking, subgraph retrieval, semantic re-ranking, and structured-to-text transformation. Unlike RAG approaches relying solely on unstructured document corpora, the system implemented here draws on the explicit relational structure of a knowledge graph to guide retrieval, enabling the capture of multi-hop connections between entities and reducing reliance on purely surface-level text matching.

From an engineering perspective, the pipeline demonstrates that symbolic graph traversal and neural retrieval techniques can be effectively combined to balance interpretability with semantic flexibility. The integration of a pretrained dense retriever into a graph-based retrieval framework yielded competitive performance, both in terms of factual correctness of the generated answers and the contextual relevance of the retrieved subgraphs. In doing so, the system bridged the gap between knowledge graph querying and LLM-based text generation, illustrating the potential for tightly coupling structured and unstructured knowledge in a single pipeline.

A Code, Artifacts and Dataset

The code is uploaded to the GitHub repository: <https://github.com/SaltyRain/subgraphrag>

All the links to the aligned dataset, intermediate and evaluation results are provided there as well.

B Generation Prompt

Listing 1 System and Generation Prompt configuration used for context fusion.

```
SYSTEM_FUSION = (  
    "You are a careful data-to-text assistant. "  
    "You merge compact knowledge snippets into a single, concise, factual context. "  
    "Preserve only information that directly helps answer the question. "  
    "Never invent facts. Deduplicate aggressively. Be terse and precise."  
)  
  
CONTEXT_FUSION_PROMPT = """You are given a question and several compact,  
head-centric snippets derived from Wikidata 1-2 hop paths.  
Merge them into ONE concise, self-contained context that maximizes  
relevance to the question.  
  
Guidelines:  
- Keep only facts that are directly helpful for answering the question.  
- Deduplicate overlapping content across snippets.  
- Prefer exact entity/relation wording; avoid speculation.  
- Be brief: target less than {max_words} words.  
- Output ONLY the final context text (no preface, no bullets, no headings).  
  
Question:  
{question}  
  
Snippets:  
{snippets}  
"""
```

C Answer Prompt

Listing 2 System and Answer Prompt configuration used in all experiments.

```
ABSTAIN_TOKEN = "NO_ANSWER"
```

```
SYSTEM_ANSWER = (  
    "You are a precise, evidence-bound extraction assistant.\n"  
    "Given a question and a short, query-focused context, return ONLY the answer  
    ↪ as:\n"  
    "- a single entity/span, or\n"  
    "- a comma-separated list of entities/spans (when multiple answers are  
    ↪ implied).\n"  
    "Rules:\n"  
    "- Extract VERBATIM spans present in the context; do NOT paraphrase or invent.\n"  
    "- If the context is insufficient or contradictory, output exactly: " +  
    ↪ ABSTAIN_TOKEN + "\n"  
    "- No prefacing text, no explanations, no quotes, no brackets, no bullets.\n"  
)
```

```
ANSWER_PROMPT = ""Question:
```

```
{question}
```

```
Context:
```

```
{context}
```

```
Output format:
```

- If exactly one answer: a single span (e.g., Nelson Mandela)
- If multiple answers: comma-separated list with a single space after each comma
↪ (e.g., A, B, C)
- If unknown/insufficient: *{abstain}*

```
Return ONLY the answer text.
```

```
""
```

Bibliography

- [Aue+07] Sören Auer et al. “DBpedia: A Nucleus for a Web of Open Data”. In: vol. 6. Jan. 2007, pp. 722–735. ISBN: 978-3-540-76297-3. DOI: 10.1007/978-3-540-76298-0_52 (cit. on p. 13).
- [Ber+13] Jonathan Berant et al. “Semantic Parsing on Freebase from Question-Answer Pairs”. In: *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*. Ed. by David Yarowsky et al. Seattle, Washington, USA: Association for Computational Linguistics, Oct. 2013, pp. 1533–1544. URL: <https://aclanthology.org/D13-1160/> (cit. on p. 56).
- [Bla23] Vladimir Blagojević. *Enhancing RAG Pipelines in Haystack: Introducing DiversityRanker and LostInTheMiddleRanker*. <https://towardsdatascience.com/enhancing-rag-pipelines-in-haystack-45f14e2bc9f5>. Accessed: 2025-08-04. 2023 (cit. on pp. 12, 17).
- [Bol+08] Kurt Bollacker et al. “Freebase: a collaboratively created graph database for structuring human knowledge”. In: *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*. SIGMOD ’08. Vancouver, Canada: Association for Computing Machinery, 2008, pp. 1247–1250. ISBN: 9781605581026. DOI: 10.1145/1376616.1376746. URL: <https://doi.org/10.1145/1376616.1376746> (cit. on p. 13).
- [Bro+20] Tom B. Brown et al. *Language Models are Few-Shot Learners*. 2020. arXiv: 2005.14165 [cs.CL]. URL: <https://arxiv.org/abs/2005.14165> (cit. on pp. 1, 9).
- [Cho+21] Shivani Choudhary et al. “A Survey of Knowledge Graph Embedding and Their Applications”. In: *CoRR* abs/2107.07842 (2021). arXiv: 2107.07842. URL: <https://arxiv.org/abs/2107.07842> (cit. on p. 19).

- [Cho+22] Aakanksha Chowdhery et al. *PaLM: Scaling Language Modeling with Pathways*. 2022. arXiv: 2204.02311 [cs.CL]. URL: <https://arxiv.org/abs/2204.02311> (cit. on p. 1).
- [Die+17] Dennis Diefenbach et al. “Question Answering Benchmarks for Wikidata”. In: *ISWC 2017*. Vienne, Austria, Oct. 2017. URL: <https://hal.science/hal-01637141> (cit. on p. 57).
- [Edg+25] Darren Edge et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. 2025. arXiv: 2404.16130 [cs.CL]. URL: <https://arxiv.org/abs/2404.16130> (cit. on pp. 13, 18).
- [Es+25] Shahul Es et al. *Ragas: Automated Evaluation of Retrieval Augmented Generation*. 2025. arXiv: 2309.15217 [cs.CL]. URL: <https://arxiv.org/abs/2309.15217> (cit. on pp. 3, 4, 15, 53).
- [FS10] Paolo Ferragina and Ugo Scaiella. “TAGME: on-the-fly annotation of short text fragments (by wikipedia entities)”. In: *Proceedings of the 19th ACM International Conference on Information and Knowledge Management*. CIKM ’10. Toronto, ON, Canada: Association for Computing Machinery, 2010, pp. 1625–1628. ISBN: 9781450300995. DOI: 10.1145/1871437.1871689. URL: <https://doi.org/10.1145/1871437.1871689> (cit. on p. 23).
- [Gal+24] Mikhail Galkin et al. *Towards Foundation Models for Knowledge Graph Reasoning*. 2024. arXiv: 2310.04562 [cs.CL]. URL: <https://arxiv.org/abs/2310.04562> (cit. on p. 19).
- [Gao+22] Luyu Gao et al. *Precise Zero-Shot Dense Retrieval without Relevance Labels*. 2022. arXiv: 2212.10496 [cs.IR]. URL: <https://arxiv.org/abs/2212.10496> (cit. on p. 12).
- [Gao+24] Yunfan Gao et al. *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2024. arXiv: 2312.10997 [cs.CL]. URL: <https://arxiv.org/abs/2312.10997> (cit. on pp. 2, 11, 12, 40, 53).
- [Guo+25] Zirui Guo et al. *LightRAG: Simple and Fast Retrieval-Augmented Generation*. 2025. arXiv: 2410.05779 [cs.IR]. URL: <https://arxiv.org/abs/2410.05779> (cit. on pp. 7, 13, 14, 19, 21, 40, 41, 53).
- [Gut+25] Bernal Jiménez Gutiérrez et al. *HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models*. 2025. arXiv: 2405.14831 [cs.CL]. URL: <https://arxiv.org/abs/2405.14831> (cit. on pp. 13, 19).

- [Guu+20] Kelvin Guu et al. “REALM: Retrieval-Augmented Language Model Pre-Training”. In: *CoRR* abs/2002.08909 (2020). arXiv: 2002.08909. URL: <https://arxiv.org/abs/2002.08909> (cit. on p. 1).
- [He+24] Xiaoxin He et al. *G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering*. 2024. arXiv: 2402.07630 [cs.LG]. URL: <https://arxiv.org/abs/2402.07630> (cit. on p. 19).
- [Hu+24] Yuntong Hu et al. *GRAG: Graph Retrieval-Augmented Generation*. 2024. arXiv: 2405.16506 [cs.LG]. URL: <https://arxiv.org/abs/2405.16506> (cit. on p. 1).
- [Hua+25] Lei Huang et al. “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions”. In: *ACM Transactions on Information Systems* 43.2 (Jan. 2025), pp. 1–55. ISSN: 1558-2868. DOI: 10.1145/3703155. URL: <http://dx.doi.org/10.1145/3703155> (cit. on pp. 1, 9).
- [IG20] Gautier Izacard and Edouard Grave. “Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering”. In: *CoRR* abs/2007.01282 (2020). arXiv: 2007.01282. URL: <https://arxiv.org/abs/2007.01282> (cit. on p. 12).
- [JW03] Glen Jeh and Jennifer Widom. “Scaling personalized web search”. In: *Proceedings of the 12th International Conference on World Wide Web*. WWW ’03. Budapest, Hungary: Association for Computing Machinery, 2003, pp. 271–279. ISBN: 1581136803. DOI: 10.1145/775152.775191. URL: <https://doi.org/10.1145/775152.775191> (cit. on p. 19).
- [Ji+23] Ziwei Ji et al. “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* 55.12 (Mar. 2023), pp. 1–38. ISSN: 1557-7341. DOI: 10.1145/3571730. URL: <http://dx.doi.org/10.1145/3571730> (cit. on pp. 1, 9, 12).
- [Kan+23] Nikhil Kandpal et al. *Large Language Models Struggle to Learn Long-Tail Knowledge*. 2023. arXiv: 2211.08411 [cs.CL]. URL: <https://arxiv.org/abs/2211.08411> (cit. on p. 9).
- [Kar+20] Vladimir Karpukhin et al. *Dense Passage Retrieval for Open-Domain Question Answering*. 2020. arXiv: 2004.04906 [cs.CL]. URL: <https://arxiv.org/abs/2004.04906> (cit. on pp. 10, 17).

- [Kli18] Jonas Klie. *wikimapper*. Accessed on 2025-07-13. 2018. URL: <https://github.com/jcklie/wikimapper> (cit. on p. 23).
- [LCT19] Kenton Lee, Ming-Wei Chang, and Kristina Toutanova. “Latent Retrieval for Weakly Supervised Open Domain Question Answering”. In: *CoRR* abs/1906.00300 (2019). arXiv: 1906.00300. URL: <http://arxiv.org/abs/1906.00300> (cit. on p. 1).
- [Lew+19] Mike Lewis et al. *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*. 2019. arXiv: 1910.13461 [cs.CL]. URL: <https://arxiv.org/abs/1910.13461> (cit. on p. 10).
- [Lew+21] Patrick Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. 2021. arXiv: 2005.11401 [cs.CL]. URL: <https://arxiv.org/abs/2005.11401> (cit. on pp. 1, 3, 9, 10, 12, 17, 18, 21).
- [Luo+25] Linhao Luo et al. *GFM-RAG: Graph Foundation Model for Retrieval Augmented Generation*. 2025. arXiv: 2502.01113 [cs.IR]. URL: <https://arxiv.org/abs/2502.01113> (cit. on pp. 13, 19).
- [Ma+23] Xinbei Ma et al. *Query Rewriting for Retrieval-Augmented Large Language Models*. 2023. arXiv: 2305.14283 [cs.CL]. URL: <https://arxiv.org/abs/2305.14283> (cit. on p. 12).
- [MRS08] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. USA: Cambridge University Press, 2008. ISBN: 0521865719 (cit. on pp. 17, 32).
- [MK24] Costas Mavromatis and George Karypis. *GNN-RAG: Graph Neural Retrieval for Large Language Model Reasoning*. 2024. arXiv: 2405.20139 [cs.CL]. URL: <https://arxiv.org/abs/2405.20139> (cit. on p. 19).
- [Min+23] Sewon Min et al. *FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation*. 2023. arXiv: 2305.14251 [cs.CL]. URL: <https://arxiv.org/abs/2305.14251> (cit. on p. 12).
- [Min+25] Shervin Minaee et al. *Large Language Models: A Survey*. 2025. arXiv: 2402.06196 [cs.CL]. URL: <https://arxiv.org/abs/2402.06196> (cit. on pp. 1, 9).

- [Mor+20] Christopher Morris et al. “TUDataset: A collection of benchmark datasets for learning with graphs”. In: *CoRR* abs/2007.08663 (2020). arXiv: 2007.08663. URL: <https://arxiv.org/abs/2007.08663> (cit. on p. 13).
- [New10] Mark Newman. *Networks: An Introduction*. Oxford University Press, Mar. 2010. ISBN: 9780199206650. DOI: 10.1093/acprof:oso/9780199206650.001.0001. URL: <https://doi.org/10.1093/acprof:oso/9780199206650.001.0001> (cit. on pp. 17, 32).
- [Par+19] Sungchan Park et al. “A Survey on Personalized PageRank Computation Algorithms”. In: *IEEE Access* 7 (2019), pp. 163049–163062. DOI: 10.1109/ACCESS.2019.2952653 (cit. on p. 19).
- [Pen+24a] Boci Peng et al. *Graph Retrieval-Augmented Generation: A Survey*. 2024. arXiv: 2408.08921 [cs.AI]. URL: <https://arxiv.org/abs/2408.08921> (cit. on pp. 1, 13, 14).
- [Pen+24b] Wenjun Peng et al. *Large Language Model based Long-tail Query Rewriting in Taobao Search*. 2024. arXiv: 2311.03758 [cs.IR]. URL: <https://arxiv.org/abs/2311.03758> (cit. on p. 12).
- [Raf+23] Colin Raffel et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. 2023. arXiv: 1910.10683 [cs.LG]. URL: <https://arxiv.org/abs/1910.10683> (cit. on p. 10).
- [RZ09] Stephen Robertson and Hugo Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond”. In: *Foundations and Trends in Information Retrieval* 3 (Jan. 2009), pp. 333–389. DOI: 10.1561/15000000019 (cit. on pp. 10, 17).
- [SB88] Gerard Salton and Christopher Buckley. “Term-weighting approaches in automatic text retrieval”. In: *Information Processing and Management* 24.5 (1988), pp. 513–523. ISSN: 0306-4573. DOI: [https://doi.org/10.1016/0306-4573\(88\)90021-0](https://doi.org/10.1016/0306-4573(88)90021-0). URL: <https://www.sciencedirect.com/science/article/pii/0306457388900210> (cit. on pp. 17, 32).
- [She23] Yuanchun Shen. “SRTK: A Toolkit for Semantic-relevant Subgraph Retrieval”. In: *arXiv preprint arXiv:2305.04101* (2023) (cit. on pp. 6, 23, 26, 30, 37).

- [Shu+21] Kurt Shuster et al. “Retrieval Augmentation Reduces Hallucination in Conversation”. In: *CoRR* abs/2104.07567 (2021). arXiv: 2104.07567. URL: <https://arxiv.org/abs/2104.07567> (cit. on p. 12).
- [SG18] Daniil Sorokin and Iryna Gurevych. “Modeling Semantics with Gated Graph Neural Networks for Knowledge Base Question Answering”. In: *Proceedings of the 27th International Conference on Computational Linguistics*. Santa Fe, New Mexico, USA: Association for Computational Linguistics, 2018, pp. 3306–3317. URL: <http://aclweb.org/anthology/C18-1280> (cit. on pp. 4, 6, 21, 38, 56).
- [Sun+18] Haitian Sun et al. “Open Domain Question Answering Using Early Fusion of Knowledge Bases and Text”. In: *CoRR* abs/1809.00782 (2018). arXiv: 1809.00782. URL: <http://arxiv.org/abs/1809.00782> (cit. on p. 19).
- [Tou+23] Hugo Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023. arXiv: 2302.13971 [cs.CL]. URL: <https://arxiv.org/abs/2302.13971> (cit. on p. 1).
- [VK14] Denny Vrandečić and Markus Krötzsch. “Wikidata: a free collaborative knowledgebase”. In: *Commun. ACM* 57.10 (Sept. 2014), pp. 78–85. ISSN: 0001-0782. DOI: 10.1145/2629489. URL: <https://doi.org/10.1145/2629489> (cit. on pp. 2, 13, 21).
- [Yih+16] Wen-tau Yih et al. “The Value of Semantic Parse Labeling for Knowledge Base Question Answering”. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*. Berlin, Germany: Association for Computational Linguistics, 2016 (cit. on pp. 4, 21, 56).
- [Zha+22] Jing Zhang et al. “Subgraph Retrieval Enhanced Model for Multi-hop Knowledge Base Question Answering”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Smaranda Muresan, Preslav Nakov, and Aline Villavicencio. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 5773–5784. DOI: 10.18653/v1/2022.acl-long.396. URL: <https://aclanthology.org/2022.acl-long.396/> (cit. on pp. 2, 4, 6, 15, 17, 21, 26).
- [Zhe+23] Lianmin Zheng et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. 2023. arXiv: 2306.05685 [cs.CL]. URL: <https://arxiv.org/abs/2306.05685> (cit. on p. 15).

Eidesstattliche Erklärung

Hiermit versichere ich, dass ich diese Masterarbeit selbstständig und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel angefertigt habe und alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, als solche gekennzeichnet sind, sowie, dass ich die Masterarbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Passau, August 18, 2025

A handwritten signature in black ink, consisting of a series of loops and a long horizontal stroke extending to the right.

Timur Garipov